Examen DevOps

Déploiement de l'application Bijouterie

Kubernetes, Jenkins et Argo CD

Étudiant	Papa Malick NDIAYE
Groupe	Travail individuel
Dépôt Git	https://github.com/pa-malick/bijouterie-microservices
Docker Hub	https://hub.docker.com/u/pamalick
Date	Novembre 2026

Sommaire

1. Introduction 3
1.1 Objectifs 3
1.2 Architecture 3
2. Prise en main locale avec docker compose 4
3. Manifests Kubernetes 5
3.1 ConfigMap ou Secret 5
3.2 envFrom et valueFrom 5
3.3 DNS interne et types de Services 6
3.4 Ressources, sondes et images 6
3.5 Bonus : volume persistant et init.sql 6
4. Tests du déploiement Kubernetes 10
5. Pipeline Jenkins 14
5.1 Installation et configuration 14
5.2 Le Jenkinsfile 16
5.3 Résultats 17
6. Argo CD 21
6.1 Installation 21
6.2 L'Application 21
6.3 Démonstrations 22
7. Sécurité 26
8. Difficultés rencontrées 26
9. Conclusion 27

1. Introduction

L'application Bijouterie est fournie sous forme de trois microservices : un frontend Nginx, une API Node.js et une base MySQL. Le travail consiste à la déployer sur Kubernetes, puis à automatiser sa construction et son déploiement.

1.1 Objectifs

- Décrire le déploiement dans des manifests Kubernetes, avec une configuration externalisée.
- Construire et publier les images Docker avec un pipeline Jenkins.
- Déployer en GitOps avec Argo CD : le dépôt Git est la seule source de vérité.

1.2 Architecture

Le développeur pousse ses commits sur GitHub. Jenkins construit les trois images, les publie sur Docker Hub, puis met à jour le tag des images dans les manifests. Argo CD surveille le dossier k8s/ et applique chaque changement au cluster.

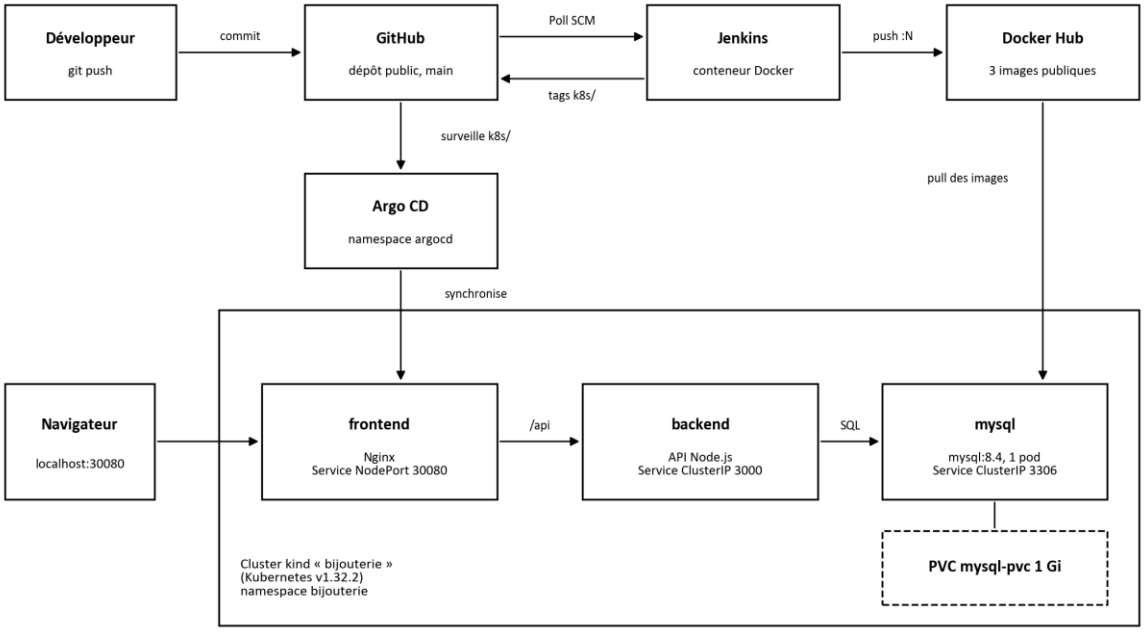


Figure 1. Architecture du déploiement et chaîne CI/CD.

Outil	Version	Rôle
Docker Desktop	moteur 28.0.4	Construction des images, exécution de kind et de Jenkins
kind	nœud v1.32.2	Cluster Kubernetes local à un nœud
Jenkins	2.580.2 (lts-jdk21)	Intégration continue
Argo CD	3.5.6	Déploiement continu GitOps

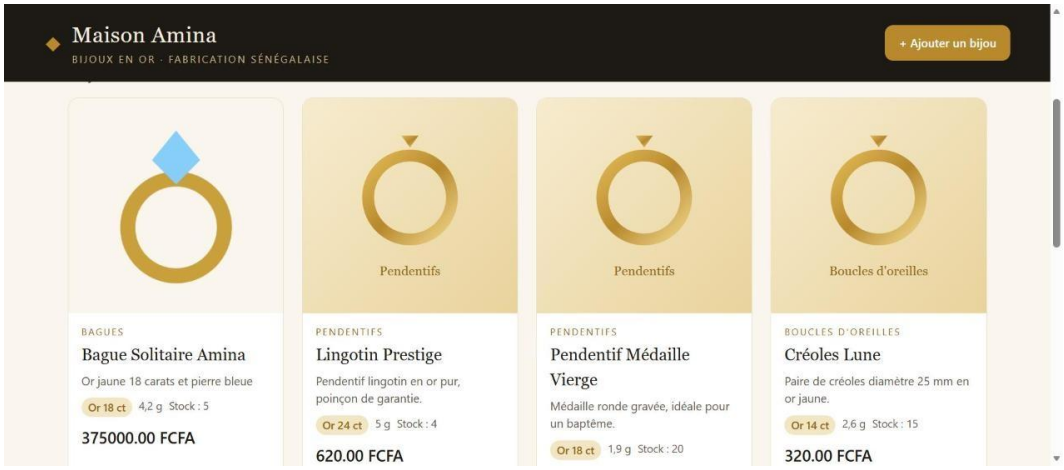
Le nœud kind est épinglé en v1.32.2 : les versions plus récentes ne démarrent pas sur ce poste, dont le Docker utilise encore les cgroups v1. Le fichier kind-config.yaml publie aussi le port 30080 du nœud sur la machine.

2. Prise en main locale avec docker compose

La commande docker compose up --build lance les trois conteneurs. Le site est servi sur http://localhost:8080. Les valeurs personnelles sont dans le fichier .env, exclu du dépôt par le .gitignore : boutique « Maison Amina », devise FCFA.

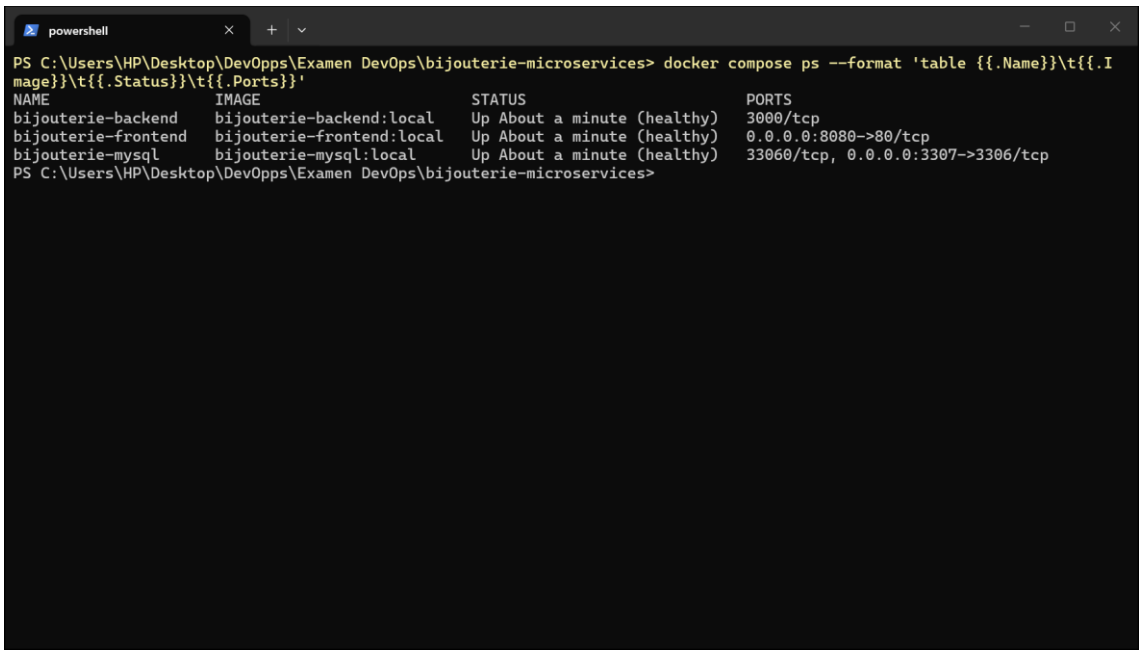
Trois points guident la suite :

- Le frontend joint l'API par BACKEND_URL = http://backend:3000. Nginx remplace cette variable au démarrage dans default.conf.template. Le nom backend est résolu par le DNS interne de Docker.
- Le backend ne publie aucun port : il n'est joignable que par le frontend.
- Chaque service reçoit sa configuration par des variables d'environnement. Dans Kubernetes, elles viendront de ConfigMaps et de Secrets.



Capture 1a. Le site en local, avec le bijou ajouté.

Elle montre le catalogue servi par docker compose, avec la bague ajoutée, sa photo et ses valeurs modifiées. Elle prouve que les trois services communiquent : l'ajout passe par l'API et est enregistré dans MySQL.



Capture 1b. Résultat de docker compose ps.

Elle montre les trois conteneurs à l'état healthy. Seuls le frontend (8080) et MySQL (3307, pour l'inspection) publient un port. Elle prouve que le backend n'est pas exposé.

3. Manifests Kubernetes

Le dossier k8s/ contient 13 fichiers numérotés, un par ressource, sans regroupement. Tous déclarent metadata.namespace: bijouterie et les labels app: bijouterie et tier (database, backend ou frontend). Chaque fichier est commenté.

Seule exception : 00-namespace.yaml n'a pas de metadata.namespace. Un Namespace est une ressource du cluster entier, il ne peut pas être rangé dans un autre namespace.

Fichier	Contenu
00-namespace.yaml	Namespace bijouterie
01-mysql-secret.yaml	MYSQL_ROOT_PASSWORD, MYSQL_DATABASE, MYSQL_USER, MYSQL_PASSWORD
02-mysql-deployment.yaml	MySQL, 1 replica, stratégie Recreate, volume persistant
03-mysql-service.yaml	Service ClusterIP mysql, port 3306
04-backend-configmap.yaml	PORT, DB_HOST, DB_PORT, DB_NAME, SHOP_NAME, CURRENCY, MAX_FILE_SIZE_MB
05-backend-secret.yaml	DB_USER et DB_PASSWORD
06-backend-deployment.yaml	API, 2 replicas, sondes /api/health et /api/ready
07-backend-service.yaml	Service ClusterIP backend, port 3000
08-frontend-configmap.yaml	BACKEND_URL et MAX_UPLOAD_SIZE
09-frontend-deployment.yaml	Nginx, 3 replicas (2 au départ), sonde /healthz
10-frontend-service.yaml	Service NodePort frontend, port 80, nodePort 30080
11-mysql-pvc.yaml	Bonus : PersistentVolumeClaim de 1 Gi
12-mysql-initdb-configmap.yaml	Bonus : script init.sql

Le frontend a été déployé avec 2 replicas, comme demandé. Il est passé à 3 par un commit, pour la démonstration GitOps du § 6.3 (captures 14a à 14c).

3.1 ConfigMap ou Secret

Une valeur non sensible va dans une ConfigMap : nom de la boutique, devise, hôte et port de la base. Une valeur sensible va dans un Secret : identifiants et mots de passe. Les deux finissent en variables d'environnement, mais un Secret peut être protégé à part (droits d'accès, chiffrement au repos). Aucune valeur n'est écrite en dur dans un Deployment.

Le compte applicatif est le même des deux côtés : MYSQL_USER et MYSQL_PASSWORD dans mysql-secret, DB_USER et DB_PASSWORD dans backend-secret.

3.2 envFrom et valueFrom

Le Deployment du backend utilise les deux formes (extrait) :

```
envFrom:                      # toutes les clés de la ConfigMap
  - configMapRef:
      name: backend-config
env:                          # une seule clé, choisie (même chose pour DB_USER)
  - name: DB_PASSWORD
    valueFrom:
      secretKeyRef:
        name: backend-secret
        key: DB_PASSWORD
```

envFrom importe toutes les clés d'un coup, chaque clé devenant une variable du même nom. C'est pratique pour une ConfigMap entière. valueFrom importe une clé précise : on voit dans le Deployment exactement quel secret est utilisé. MySQL et le frontend utilisent envFrom.

3.3 DNS interne et types de Services

Chaque Service reçoit un nom DNS dans le cluster. Le backend joint la base par DB_HOST=mysql, et Nginx joint l'API par http://backend:3000. Le nom complet serait backend.bijouterie.svc.cluster.local, mais le nom court suffit dans le même namespace. Ce nom reste stable alors que les pods changent d'adresse.

Service	Type	Raison
mysql	ClusterIP	La base ne doit être joignable que depuis le cluster
backend	ClusterIP	L'API n'est appelée que par le frontend
frontend	NodePort 30080	Seul point d'entrée depuis l'extérieur

```
spec:
  type: NodePort
  selector:            # le Service envoie le trafic aux pods qui portent ces labels
    app: bijouterie
    tier: frontend
  ports:
    - port: 80         # port du Service
      targetPort: 80   # port du conteneur Nginx
      nodePort: 30080  # port ouvert sur le nœud, publié par kind sur localhost
```

3.4 Ressources, sondes et images

- Chaque conteneur a des requests (réservation pour le scheduler) et des limits (plafond).
- Le backend a une livenessProbe sur /api/health (redémarrage si l'API ne répond plus) et une readinessProbe sur /api/ready (pas de trafic tant que la base n'est pas joignable).
- Les images viennent de Docker Hub avec un tag numéroté, jamais latest. Le tag est mis à jour par Jenkins.

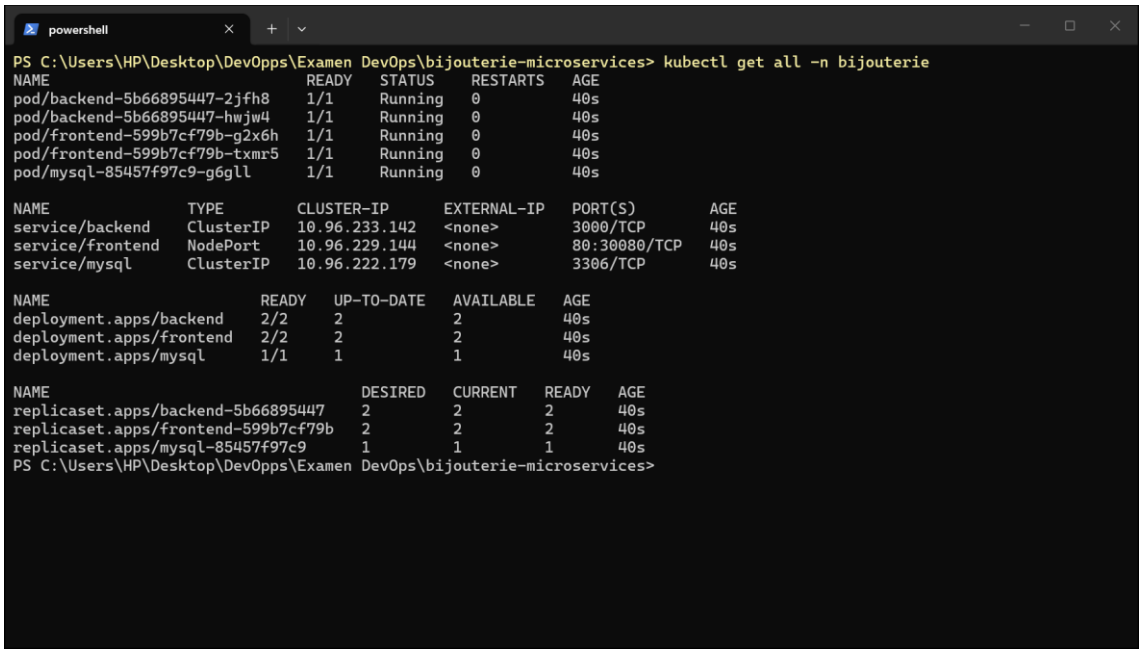
3.5 Bonus : volume persistant et init.sql

Le PVC mysql-pvc demande 1 Gi à la classe de stockage de kind. Il est monté sur /var/lib/mysql. Le script init.sql est placé dans la ConfigMap mysql-initdb et monté dans /docker-entrypoint-initdb.d : le Deployment utilise donc l'image officielle mysql:8.4.

Ce choix suit le bonus et remplace l'image pamalick/bijouterie-mysql demandée au § 4.4. Cette image personnelle a servi pour le premier déploiement et les tests du § 4.9. Jenkins continue de la construire et de la publier à chaque build.

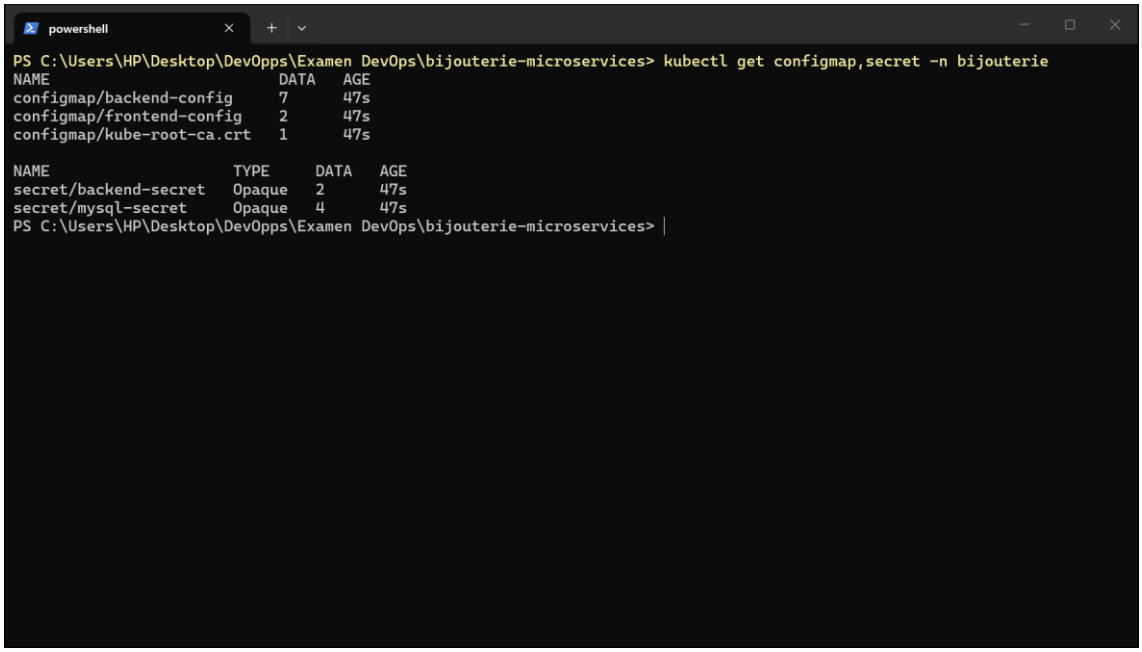
```
volumeMounts:
  - name: mysql-data             # volume rattaché au PVC mysql-pvc
    mountPath: /var/lib/mysql
  - name: mysql-initdb           # volume rempli par la ConfigMap mysql-initdb
    mountPath: /docker-entrypoint-initdb.d
```

init.sql n'est exécuté que si le dossier des données est vide, donc au tout premier démarrage.



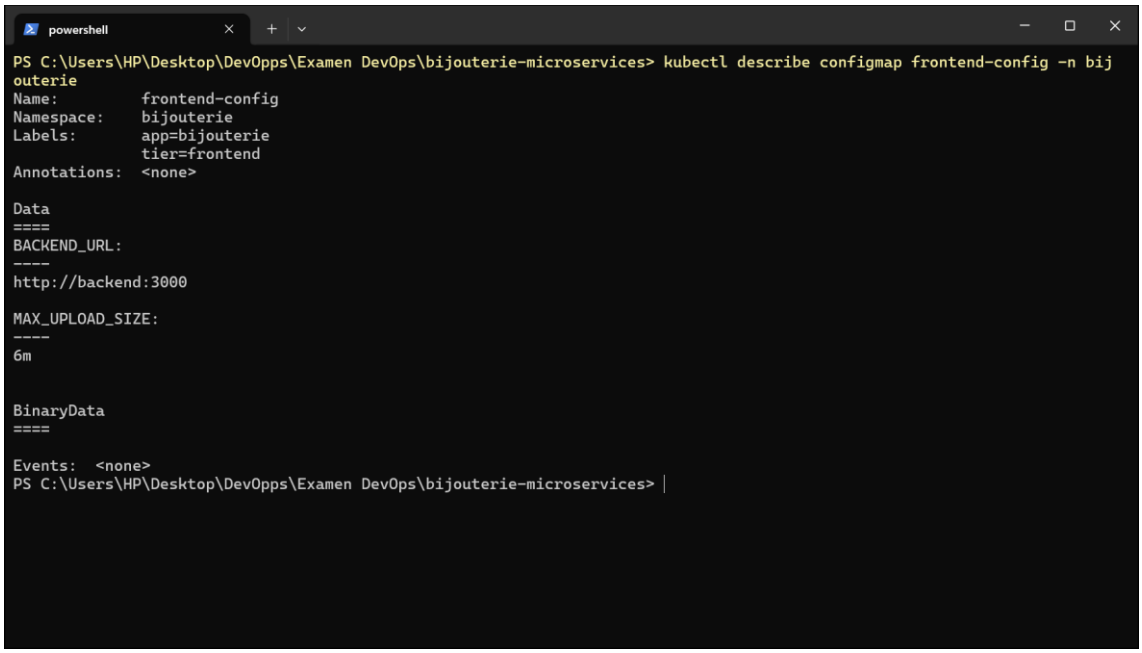
Capture 2. kubectl get all -n bijouterie.

Elle montre les 5 pods en Running, les 3 Deployments et les 3 Services. Elle prouve que le déploiement est complet et sain.



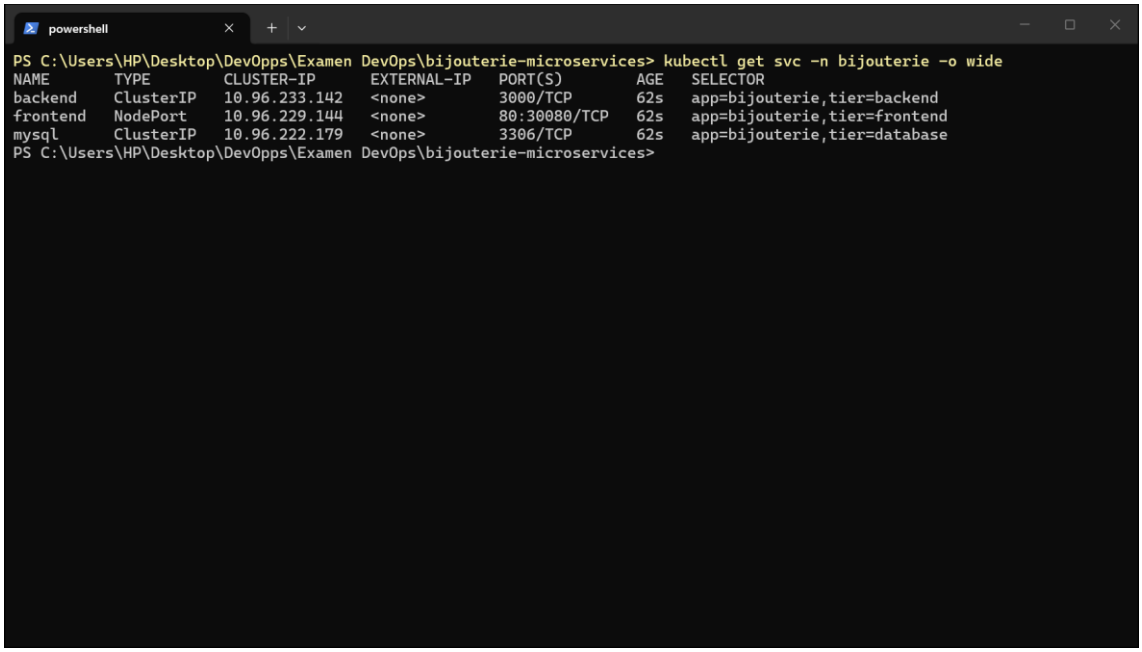
Capture 3a. kubectl get configmap,secret -n bijouterie.

Elle montre les ConfigMaps et les Secrets imposés, avec leur nombre de clés. Les valeurs des Secrets ne sont pas affichées.



Capture 3b. kubectl describe configmap frontend-config.

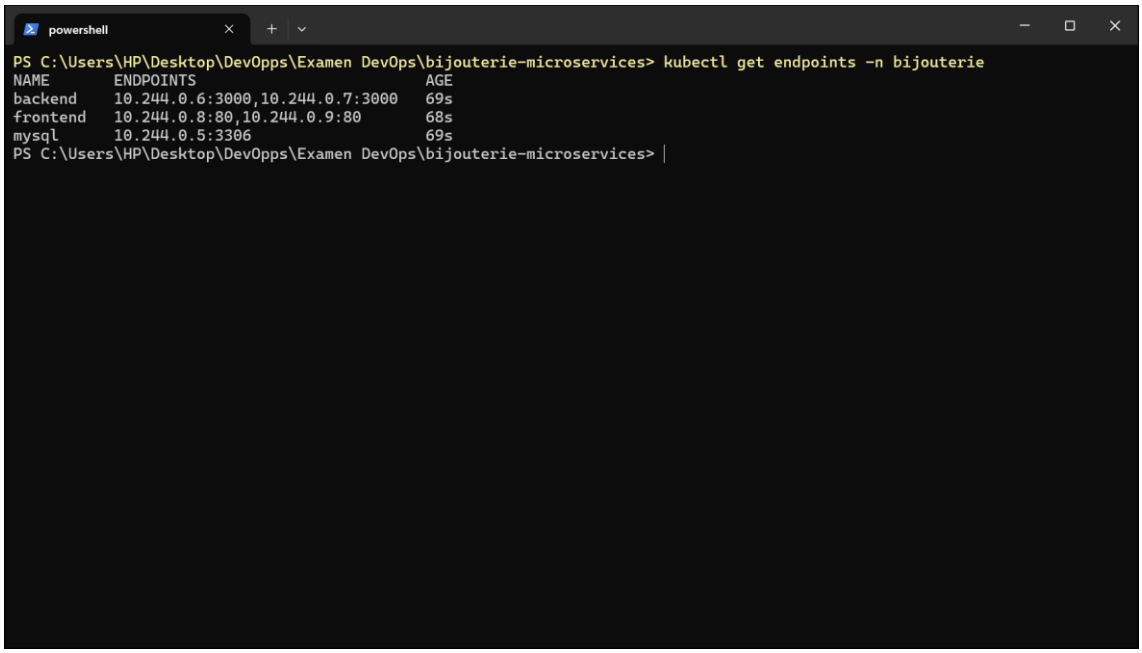
Elle montre les clés BACKEND_URL et MAX_UPLOAD_SIZE. Elle prouve que l'adresse de l'API utilise le nom DNS du Service backend.



Capture 4. kubectl get svc -n bijouterie.

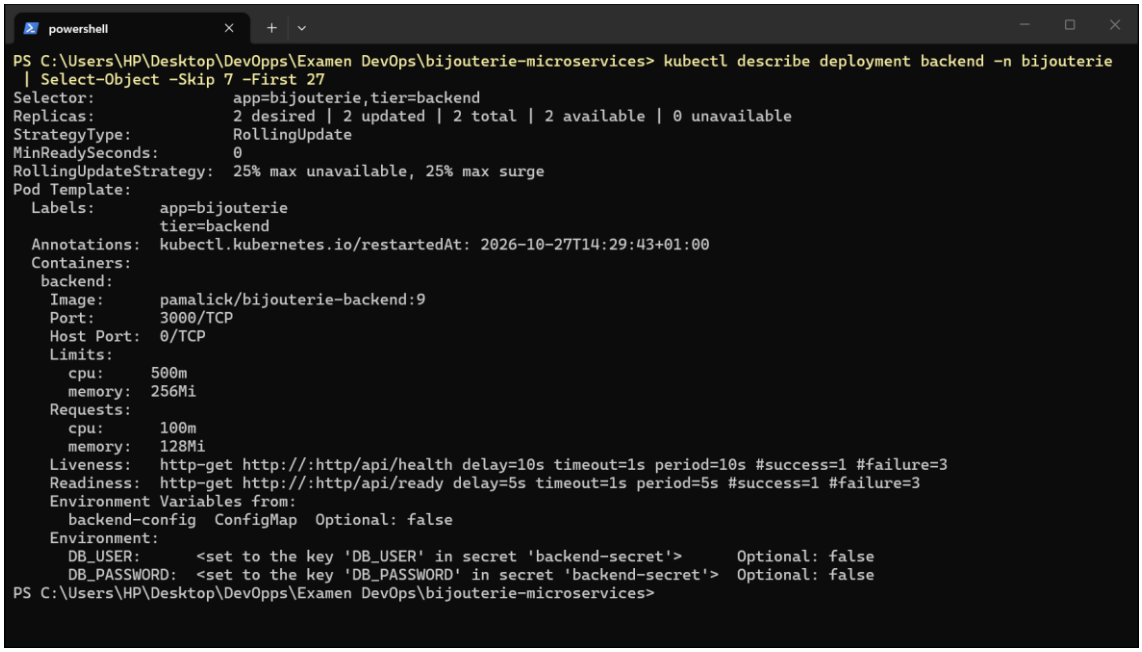
Elle montre le Service frontend en NodePort 80:30080 et les Services backend et mysql en ClusterIP. Elle prouve que seul le frontend est exposé.

Les autres commandes de vérification du § 4.8 :



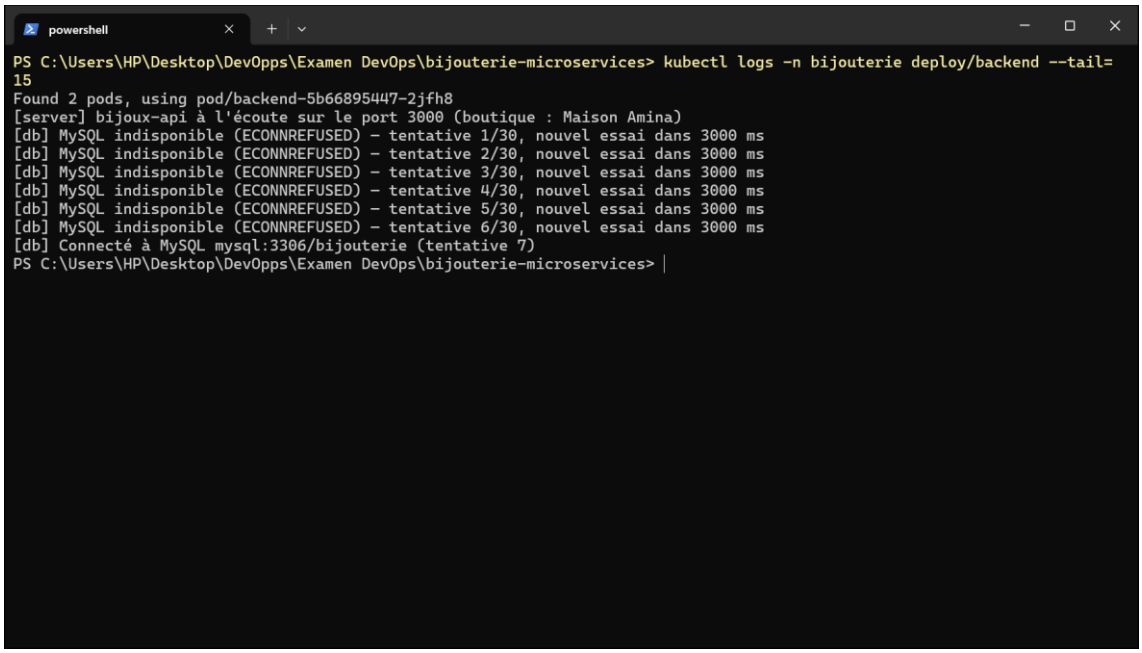
Capture 4b. kubectl get endpoints -n bijouterie.

Chaque Service liste les adresses des pods qu'il sert : 2 pour backend et frontend, 1 pour mysql. Elle prouve que les selectors des Services trouvent bien les pods par leurs labels.



Capture 4c. kubectl describe deployment backend -n bijouterie (extrait).

On y lit le selector, les 2 replicas, l'image taguée, les requests et limits et les deux sondes. La configuration vient de backend-config (envFrom) et de backend-secret (valueFrom) : seules les références sont affichées, jamais les valeurs.



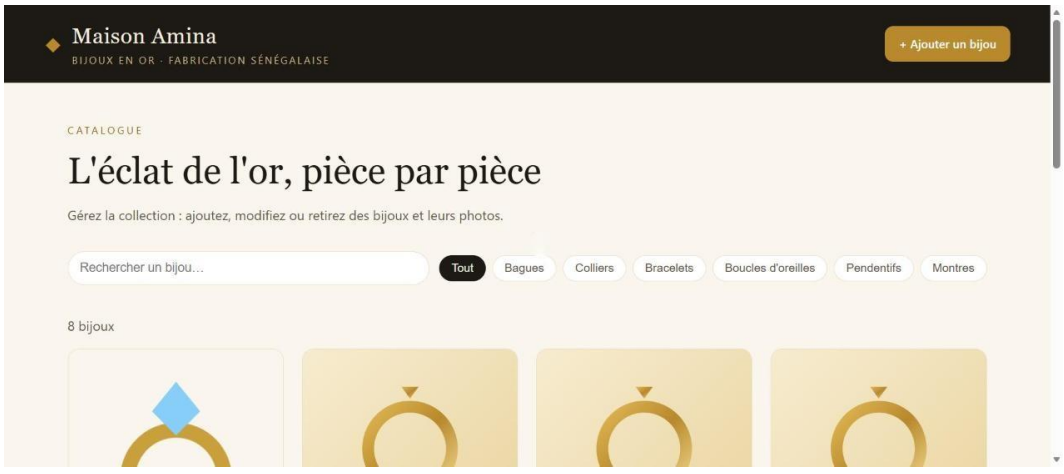
Capture 4d. kubectl logs -n bijouterie deploy/backend.

Au premier démarrage, le backend attend MySQL pendant 6 tentatives, puis se connecte à mysql:3306. Elle prouve que DB_HOST=mysql est résolu par le DNS du cluster.

4. Tests du déploiement Kubernetes

Les manifests ont été appliqués une seule fois à la main avec kubectl apply -f k8s/, avant Argo CD.

Test	Résultat
Site via le NodePort	Le site s'affiche sur localhost:30080 avec le nom Maison Amina
Ajout, modification, suppression	Bague ajoutée avec photo, prix et stock modifiés, un bijou supprimé
Suppression d'un pod backend	Le Deployment recrée un pod en quelques secondes
Champ servedBy	Les réponses alternent entre les 2 pods : le Service répartit la charge
Suppression du pod MySQL	Sans volume, les données ajoutées sont perdues
Changement de SHOP_NAME	Visible seulement après kubectl rollout restart



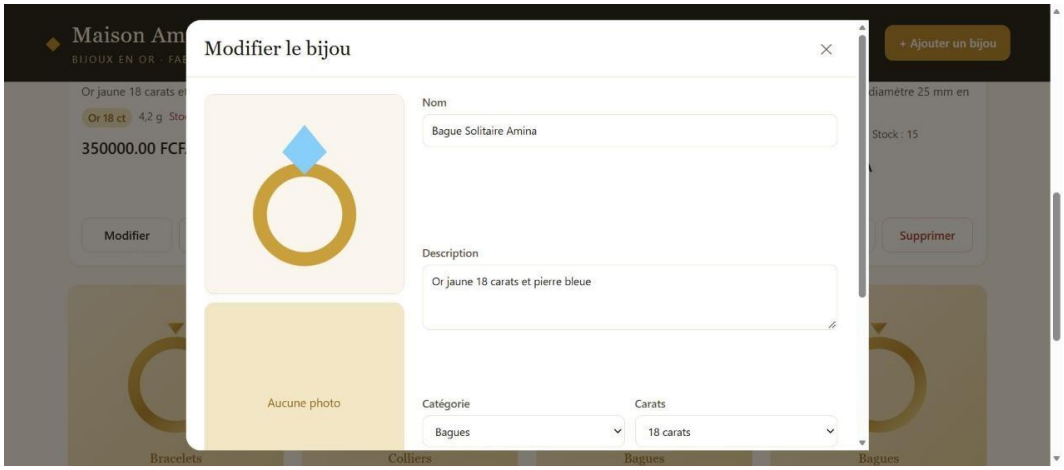
Capture 5. Le site via le NodePort 30080.

Elle montre l'en-tête « Maison Amina » servi par le cluster. Elle prouve que le chemin complet fonctionne : kind, Service NodePort, Nginx, API et base.



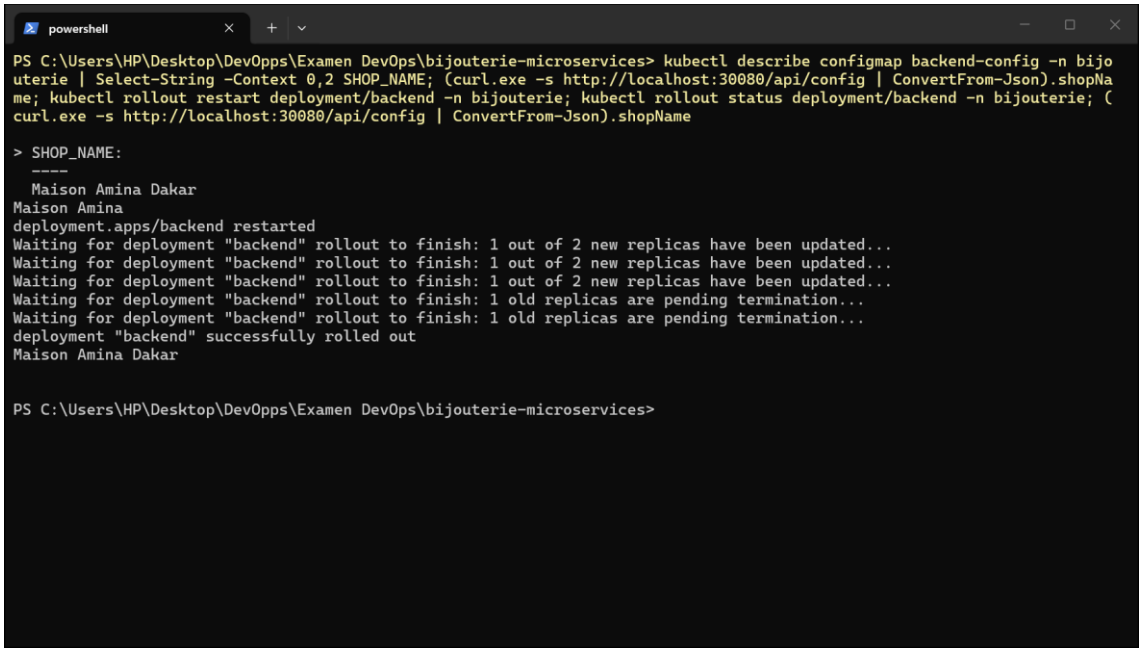
Capture 6a. Bijou ajouté avec sa photo.

La photo est envoyée par le formulaire, stockée dans MySQL et relue par l'API.



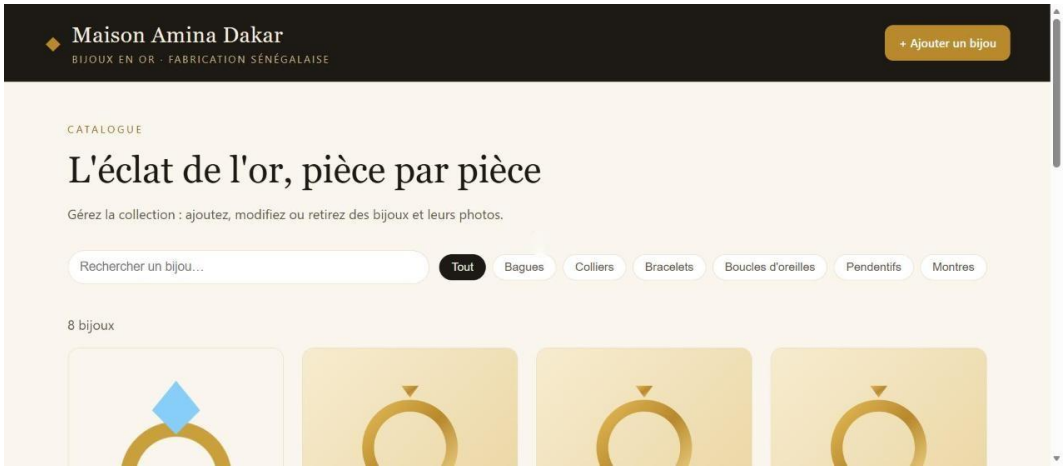
Capture 6b. Formulaire de modification.

Le formulaire est prérempli avec les valeurs lues en base.



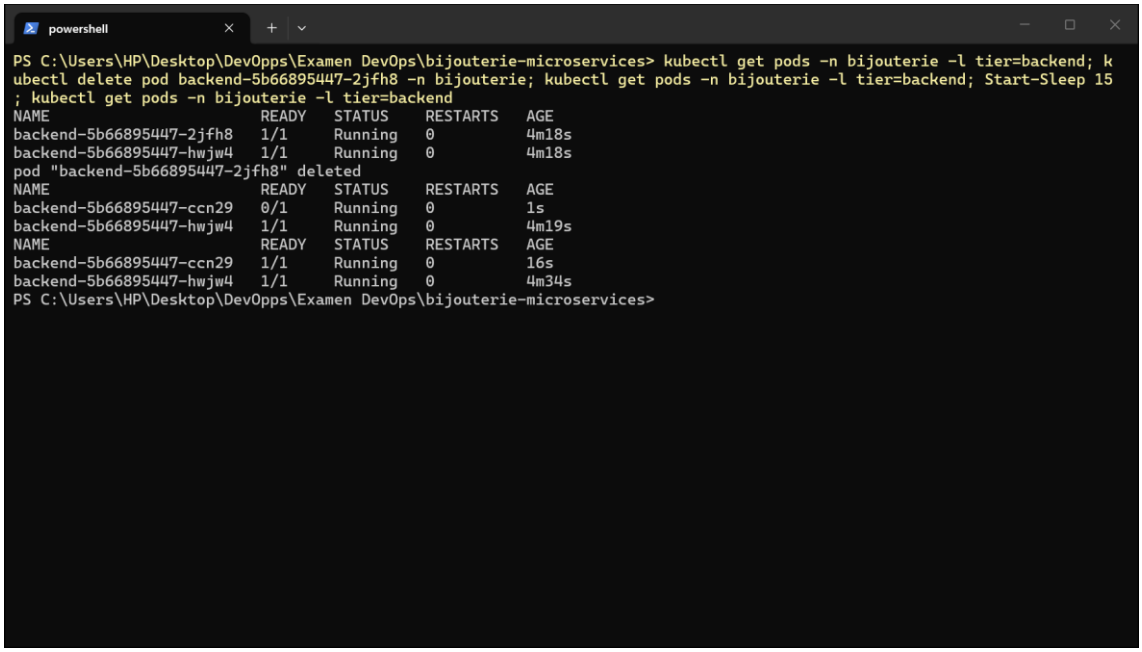
Capture 7a. Changement de SHOP_NAME et rollout restart.

La ConfigMap est modifiée, puis les pods backend sont recréés. Une variable d'environnement est lue au démarrage du conteneur : modifier la ConfigMap ne suffit pas.



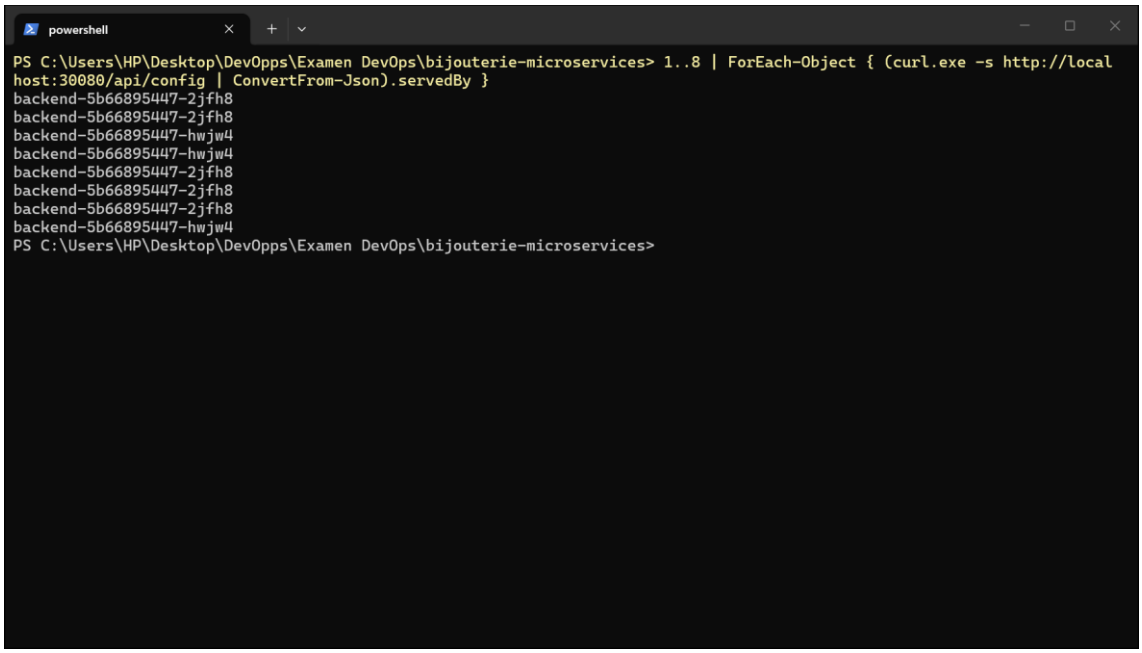
Capture 7b. Nouveau nom de boutique.

Le site affiche « Maison Amina Dakar » après le redémarrage. Le nom a ensuite été remis à Maison Amina.



Capture 8. Recréation d'un pod backend supprimé.

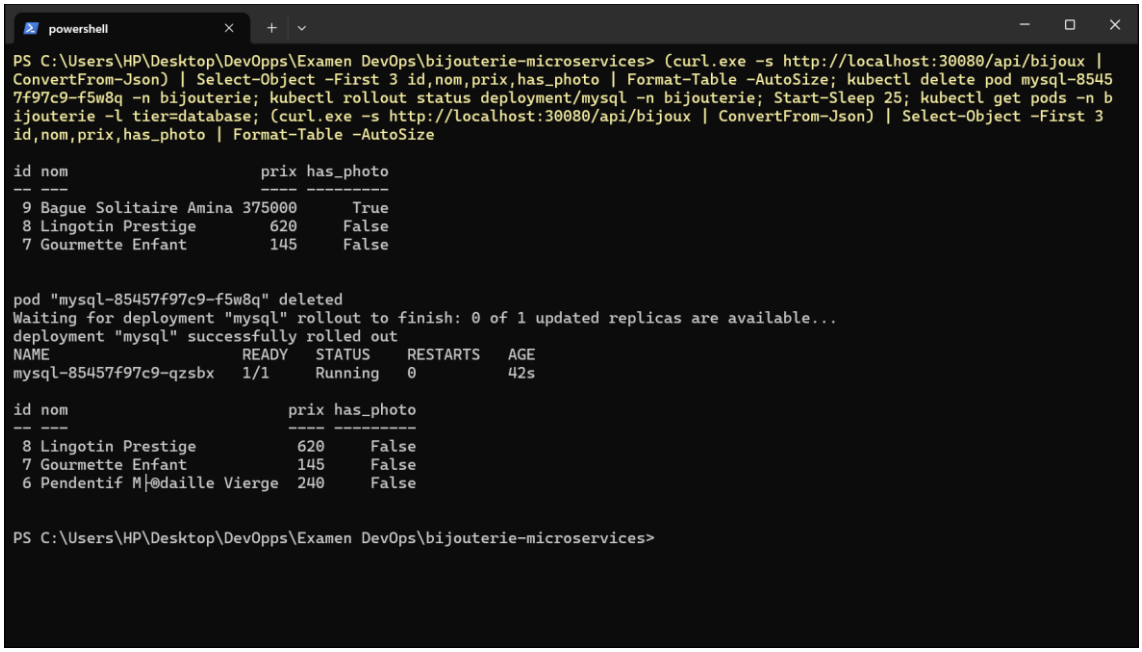
Un pod est supprimé avec kubectl delete pod, un nouveau pod apparaît aussitôt. Elle prouve que le Deployment maintient le nombre de replicas demandé.



Capture 8b. Test 4 : huit appels à /api/config.

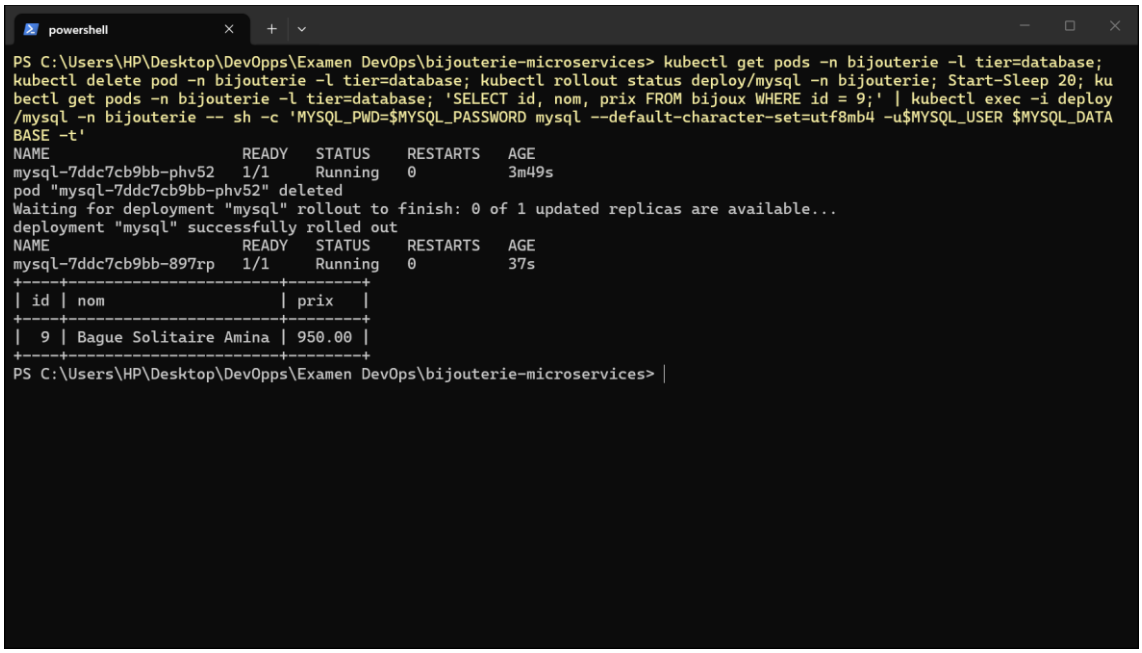
Le champ servedBy donne le nom du pod qui a répondu. Il alterne entre les deux pods backend.

C'est le rôle du Service ClusterIP backend. Il offre une adresse et un nom DNS stables devant les pods, et répartit les requêtes entre les pods prêts qui portent les labels de son selector. Le frontend appelle toujours http://backend:3000, sans savoir combien de pods existent ni lesquels. Si un pod tombe, il sort de la liste des endpoints et le trafic va vers les autres.



Capture 8c. Suppression du pod MySQL sans volume.

Le bijou n°9 disparaît après la recréation du pod. Sans volume, les données vivent dans le conteneur et meurent avec lui.



Capture 8d. Même test avec le volume persistant (bonus).

Avec le PVC, le bijou n°9 est toujours là après la suppression du pod. Elle prouve l'intérêt du volume persistant.

5. Pipeline Jenkins

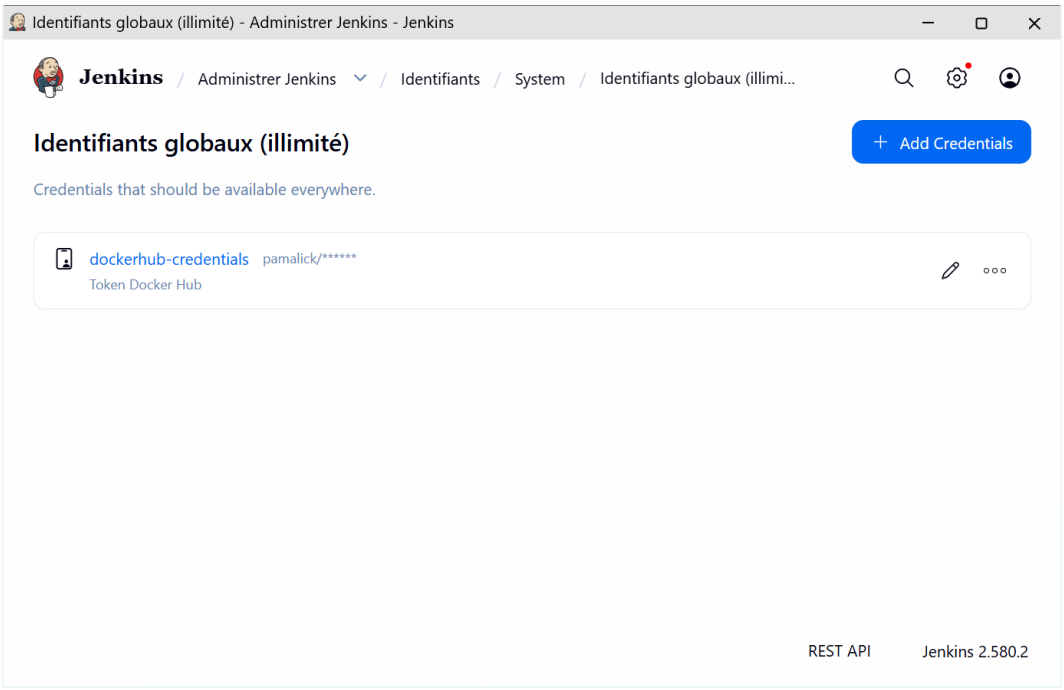
5.1 Installation et configuration

Jenkins tourne dans un conteneur Docker. L'image officielle est complétée par la CLI docker, et le socket Docker de la machine est monté : Jenkins construit ses images avec le moteur de l'hôte.

```
FROM jenkins/jenkins:lts-jdk21
USER root
COPY --from=docker:cli /usr/local/bin/docker /usr/local/bin/docker
COPY --from=docker:cli /usr/local/libexec/docker/cli-plugins/docker-buildx \
     /usr/local/libexec/docker/cli-plugins/docker-buildx
USER jenkins
```

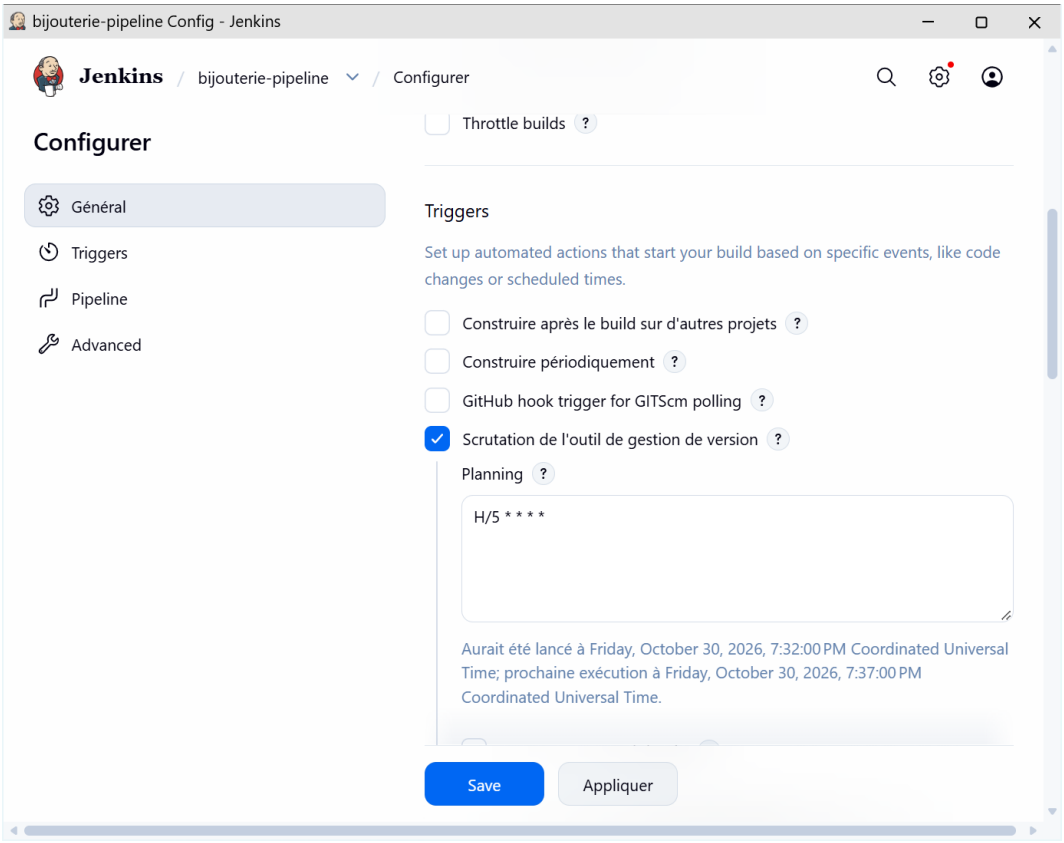
```
docker run -d --name jenkins --restart unless-stopped --group-add 0 -p 8080:8080 \
  -v jenkins_home:/var/jenkins_home -v /var/run/docker.sock:/var/run/docker.sock \
  jenkins-docker:lts-jdk21
```

- dockerhub-credentials : type Username with password, avec un Access Token Docker Hub.
- github-credentials : token GitHub limité au dépôt, permission Contents en lecture et écriture (utilisé par le stage Update manifests).
- Job bijouterie-pipeline de type Pipeline script from SCM : dépôt GitHub, branche */main, Script Path Jenkinsfile. Déclencheur Poll SCM toutes les 5 minutes.



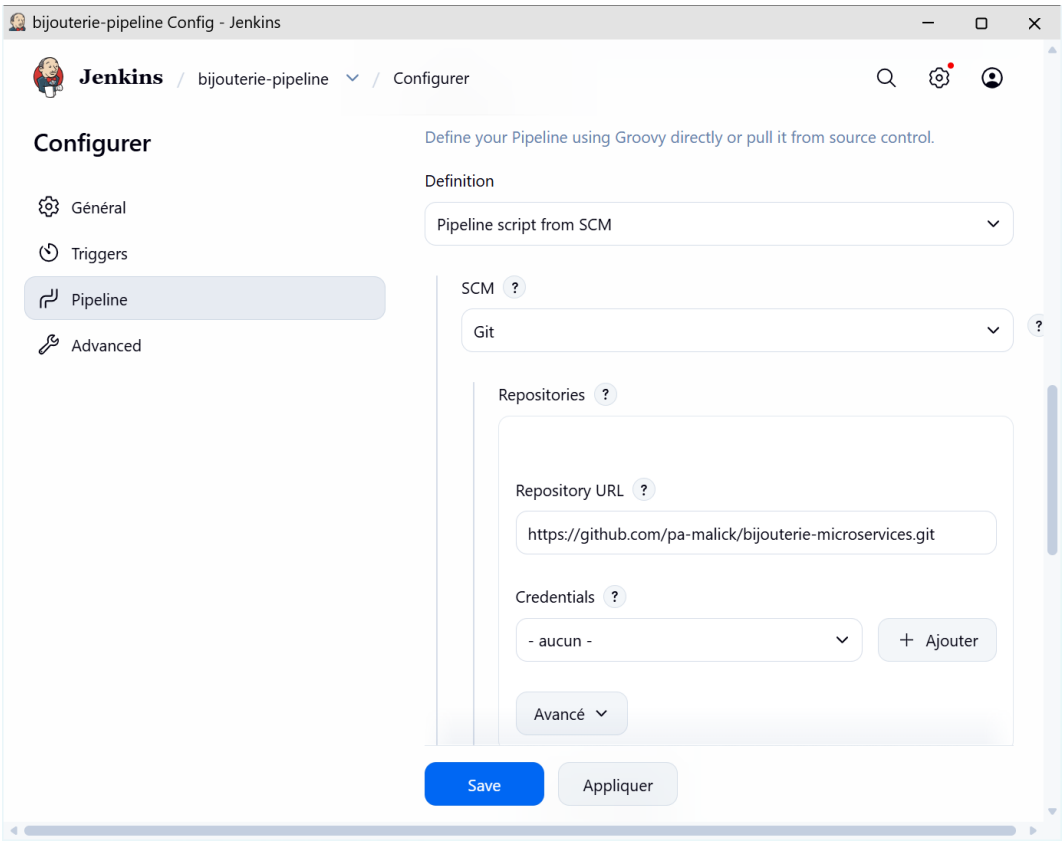
Capture 9. Le credential dockerhub-credentials.

Le token est masqué (pamalick/******). Il n'apparaît ni dans le Jenkinsfile ni dans le dépôt.



Capture 10a. Déclencheur du job.

Poll SCM avec le planning H/5 * * * * : Jenkins interroge GitHub toutes les 5 minutes.



Capture 10b. Source du pipeline.

Le Jenkinsfile est lu dans le dépôt GitHub : le pipeline est versionné avec le code.

5.2 Le Jenkinsfile

Stage	Rôle
Checkout	Récupère le code et calcule l'identifiant court du commit
Build	Construit les 3 images avec le tag BUILD_NUMBER
Tag	Ajoute un second tag : l'identifiant court du commit
Push	Se connecte à Docker Hub et pousse les deux tags des trois images
Update manifests (bonus)	Met à jour le tag dans k8s/, committe et pousse sur main
post always	docker logout, que le build réussisse ou non

Le double tag a deux usages. Le numéro de build est simple à lire et à suivre. L'identifiant du commit relie chaque image au code exact qui l'a produite.

Extrait du stage Push :

```
withCredentials([usernamePassword(credentialsId: 'dockerhub-credentials',
                                  usernameVariable: 'DOCKERHUB_USER',
                                  passwordVariable: 'DOCKERHUB_TOKEN')]) {
    sh 'echo "$DOCKERHUB_TOKEN" | docker login -u "$DOCKERHUB_USER" --password-stdin'
}
sh 'docker push $REGISTRY/bijouterie-frontend:$BUILD_NUMBER'
sh 'docker push $REGISTRY/bijouterie-frontend:$GIT_SHORT'
```

Les commandes sh sont entre apostrophes simples : c'est le shell qui lit les variables, pas Groovy. Le token n'est donc jamais inséré dans le texte du script, et Jenkins le masque dans la console.

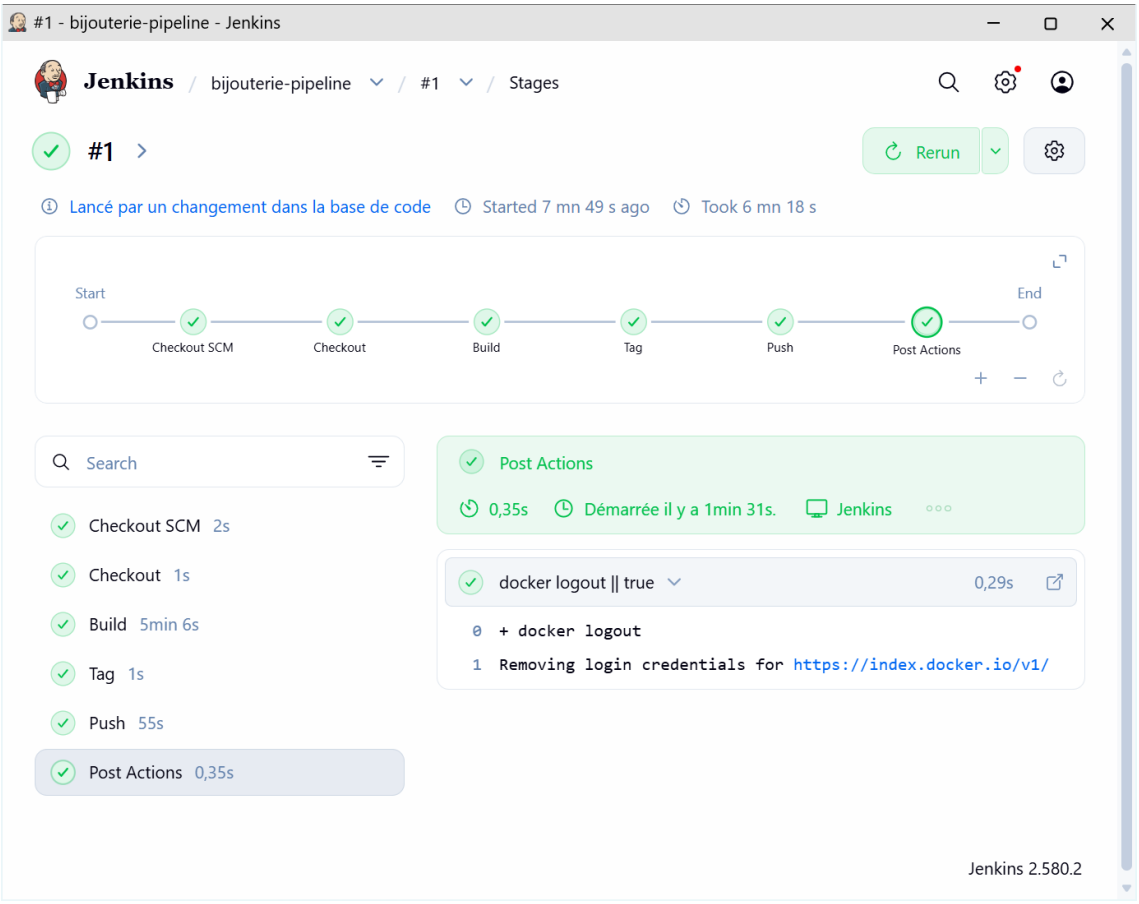
Extrait du stage Update manifests :

```
sh 'sed -i "s|$REGISTRY/bijouterie-backend:.*|$REGISTRY/bijouterie-backend:$BUILD_NUMBER|"
k8s/06-backend-deployment.yaml'
sh 'git add k8s/06-backend-deployment.yaml k8s/09-frontend-deployment.yaml'
sh 'git commit -m "Mise à jour des images : build $BUILD_NUMBER [skip ci]"'
withCredentials([usernamePassword(credentialsId: 'github-credentials',
                                  usernameVariable: 'GIT_USER',
                                  passwordVariable: 'GIT_TOKEN')]) {
    sh 'git push https://$GIT_USER:$GIT_TOKEN@$GIT_REPO HEAD:main'
}
```

sed remplace le tag de l'image (même chose pour le frontend). Jenkins committe ensuite sous son nom, puis pousse sur main avec le token GitHub.

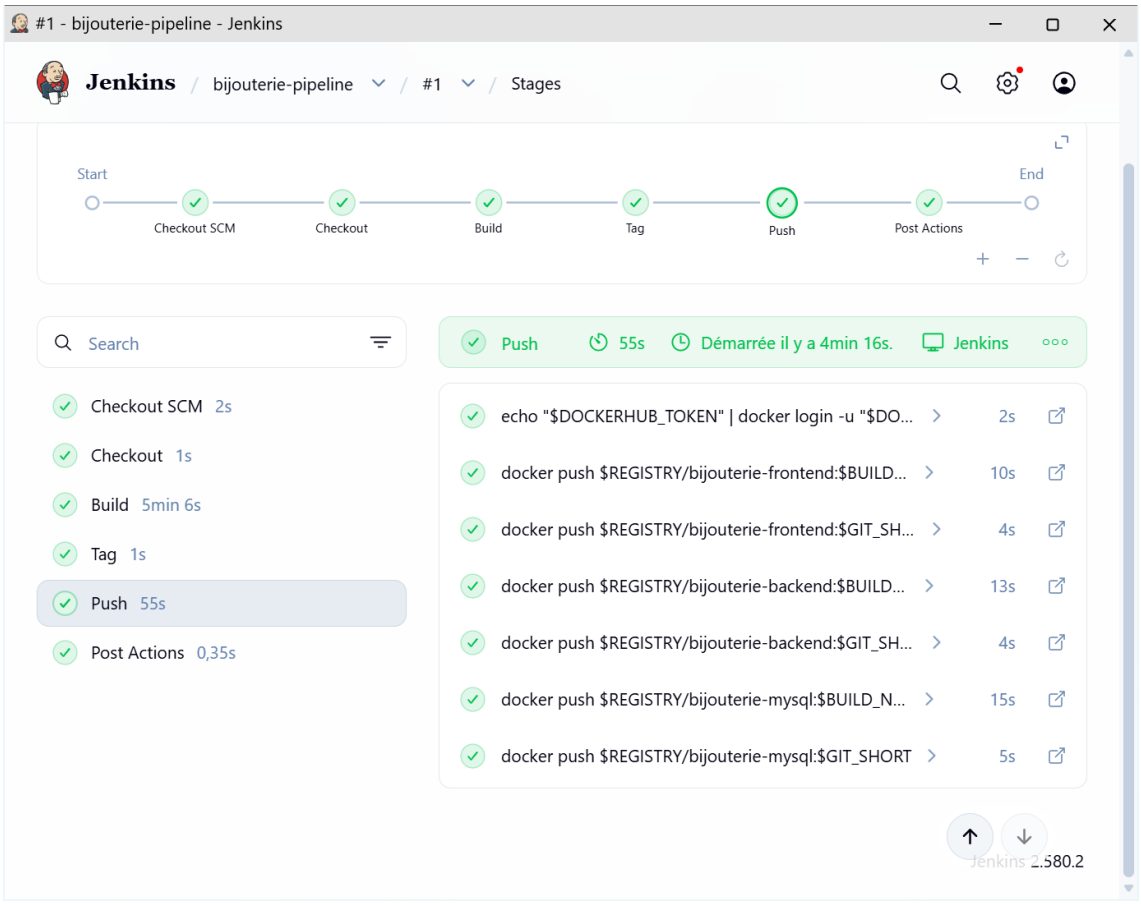
Le commit de Jenkins est lui aussi vu par le Poll SCM. Sans précaution, chaque build en lancerait un autre. Le message contient donc [skip ci] : le Checkout le détecte, et les stages suivants sont sautés grâce à la condition when. MySQL n'est pas concerné par ce stage, car il utilise l'image officielle.

5.3 Résultats



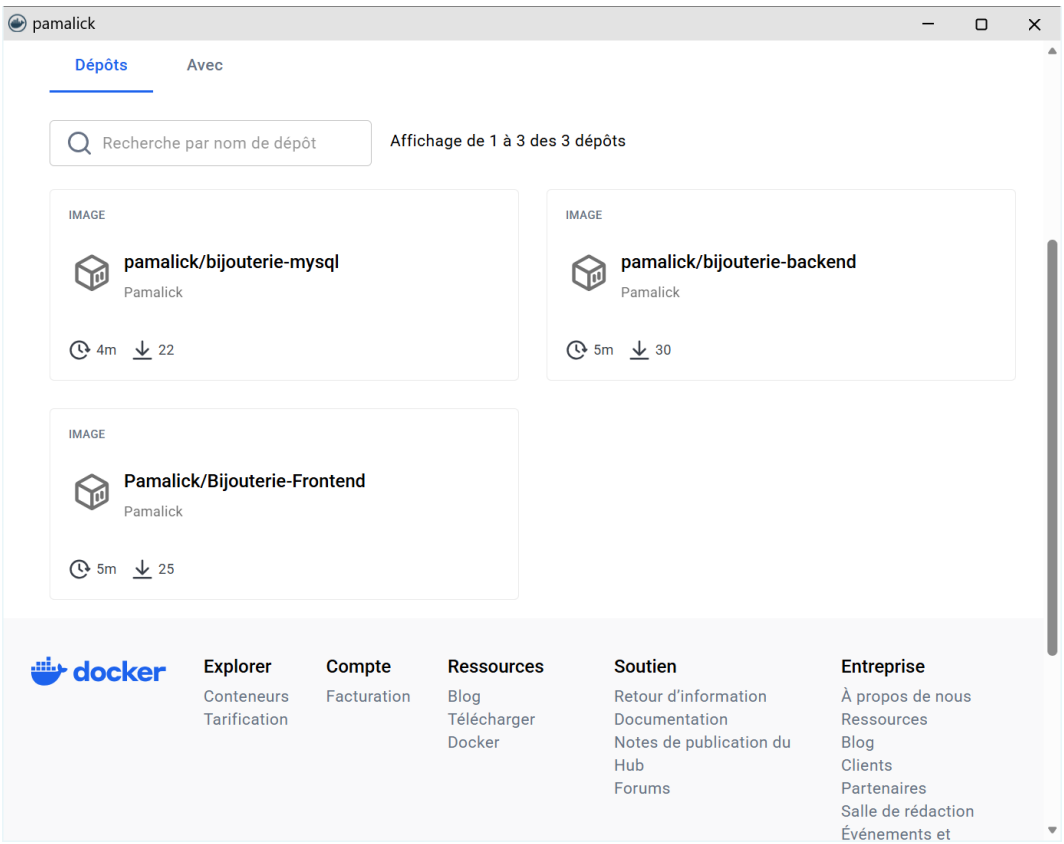
Capture 11a. Stages du build n°1.

Le build a été lancé par le Poll SCM (« Lancé par un changement dans la base de code »). Tous les stages sont réussis, en 6 min 18 s pour ce premier build.



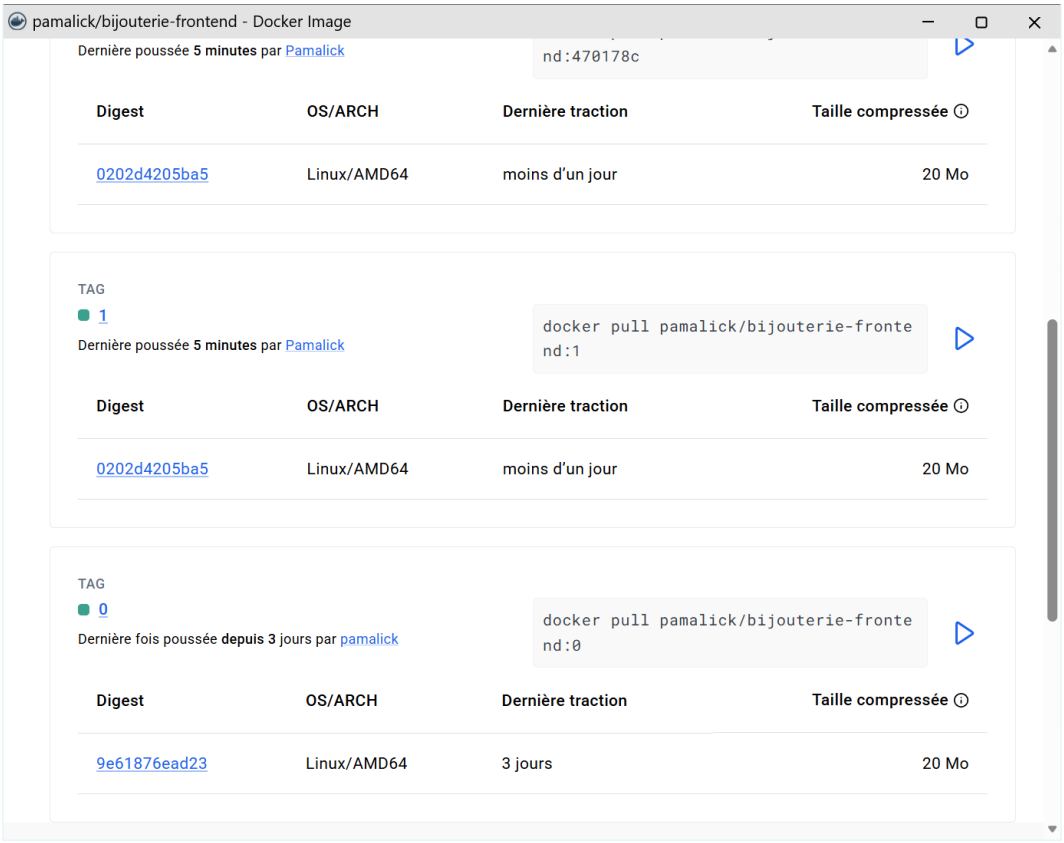
Capture 11b. Détail du stage Push.

On voit le docker login puis les six docker push. Elle prouve le double tag et l'envoi sur Docker Hub.



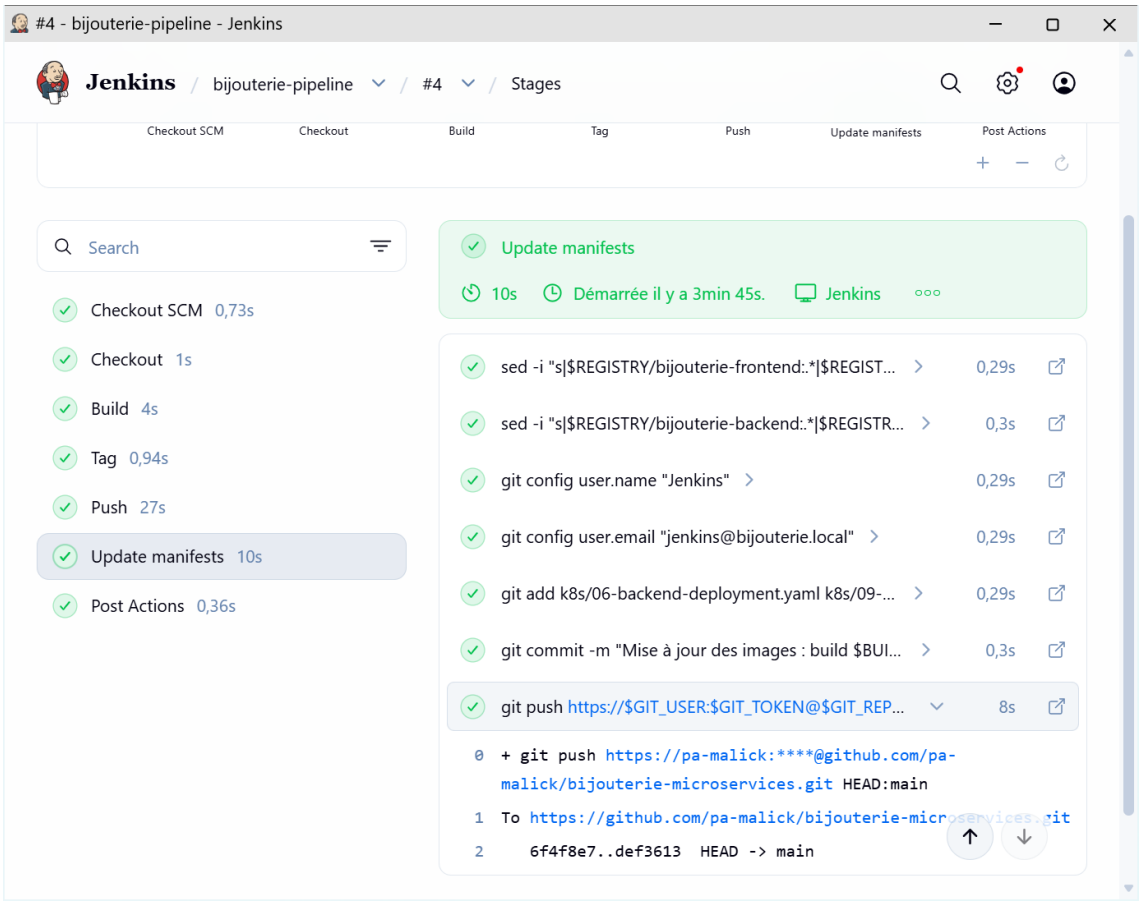
Capture 12a. Les trois dépôts Docker Hub.

Les trois images sont publiques sur le compte pamalick.



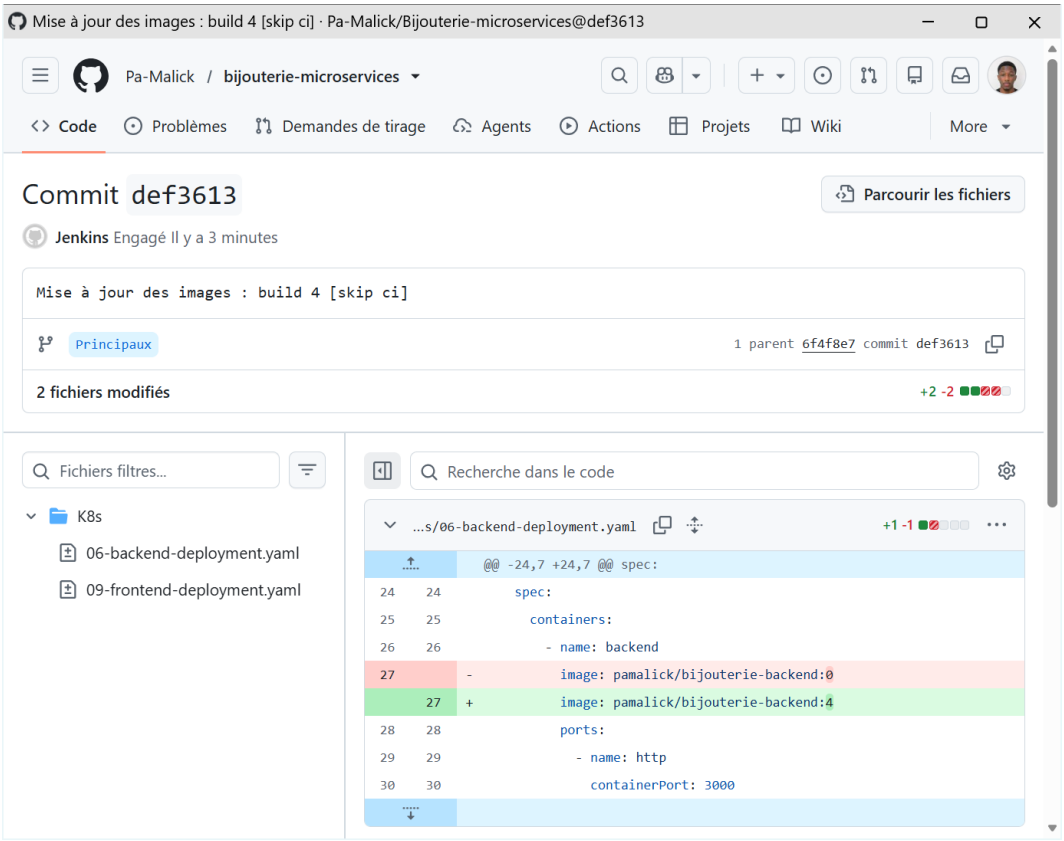
Capture 12b. Tags de l'image frontend.

Les tags 470178c et 1 ont le même digest : c'est la même image avec deux tags. Le tag 0 vient du premier push manuel.



Capture 12c. Stage Update manifests (bonus).

sed, commit puis push vers main. Le token apparaît masqué dans l'URL (****).



Capture 12d. Commit poussé par Jenkins.

Le commit de Jenkins remplace le tag 0 par 4 dans les deux Deployments.

6. Argo CD

6.1 Installation

Argo CD est installé dans le namespace argocd avec le manifeste officiel. L'option server.insecure=true sert l'interface en HTTP, ce qui suffit en local. L'interface est ouverte par kubectl port-forward sur http://localhost:8443, car le port 8080 est pris par Jenkins. La connexion utilise le compte admin.

```
kubectl create namespace argocd
kubectl apply -n argocd --server-side -f https://raw.githubusercontent.com/argoproj/argo-
cd/stable/manifests/install.yaml
kubectl port-forward svc/argocd-server -n argocd 8443:80
```

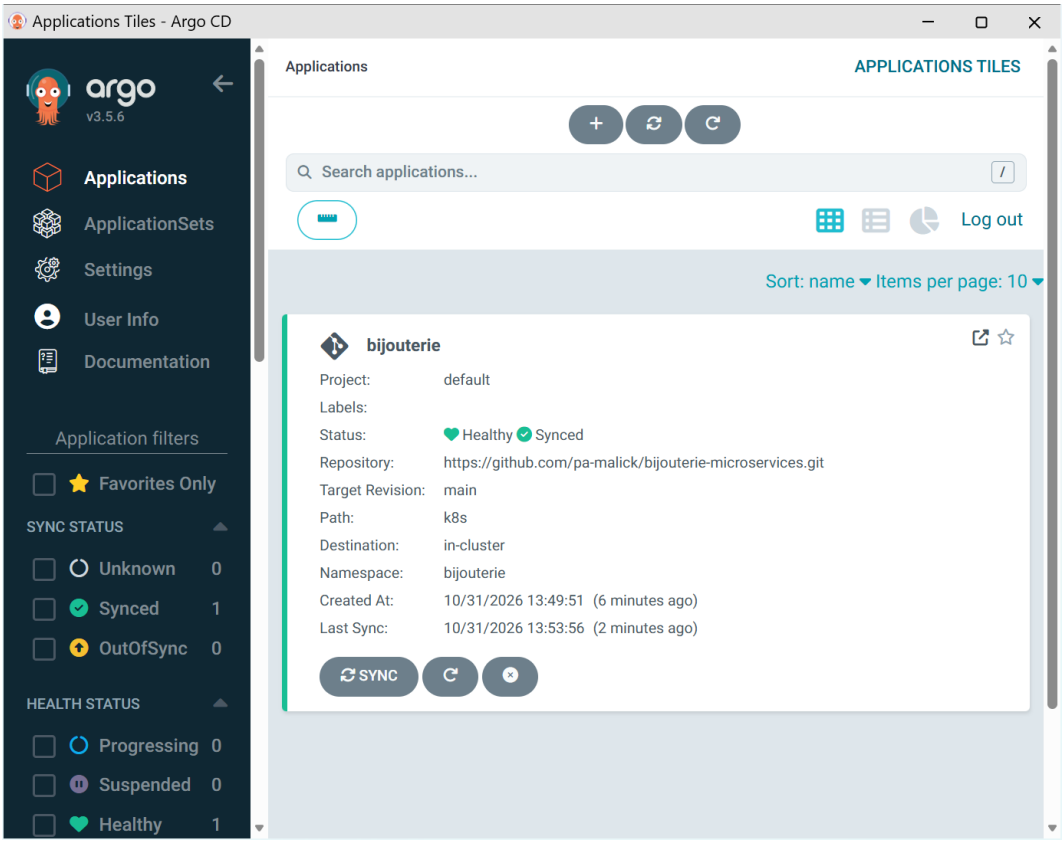
6.2 L'Application

Le fichier argocd/application.yaml déclare l'Application bijouterie dans le namespace argocd. Extrait de sa partie spec :

```
source:
  repoURL: https://github.com/pa-malick/bijouterie-microservices.git
  targetRevision: main
  path: k8s
destination:
  server: https://kubernetes.default.svc
  namespace: bijouterie
syncPolicy:
  automated:
    prune: true      # une ressource retirée de Git est supprimée du cluster
    selfHeal: true   # une modification manuelle du cluster est annulée
  syncOptions:
    - CreateNamespace=true
```

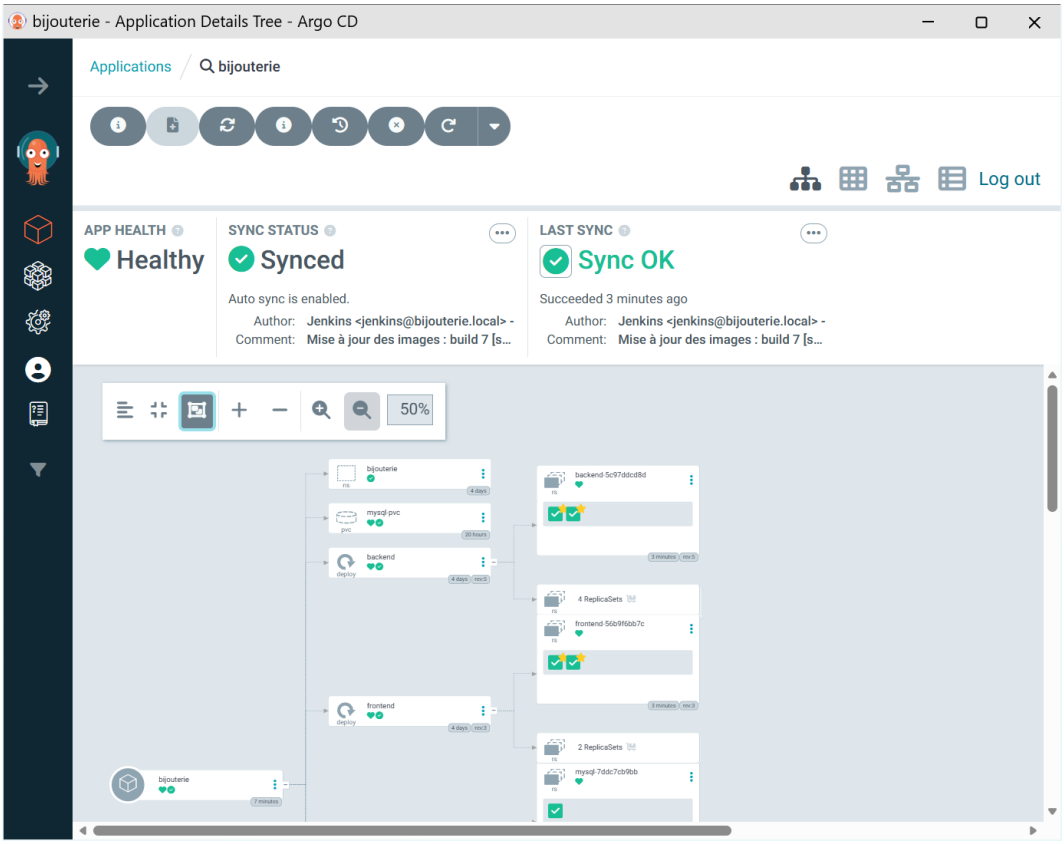
Ce fichier est la dernière ressource appliquée à la main. Depuis, aucune commande kubectl apply n'est faite sur le namespace bijouterie : tout changement passe par un commit.

6.3 Démonstrations



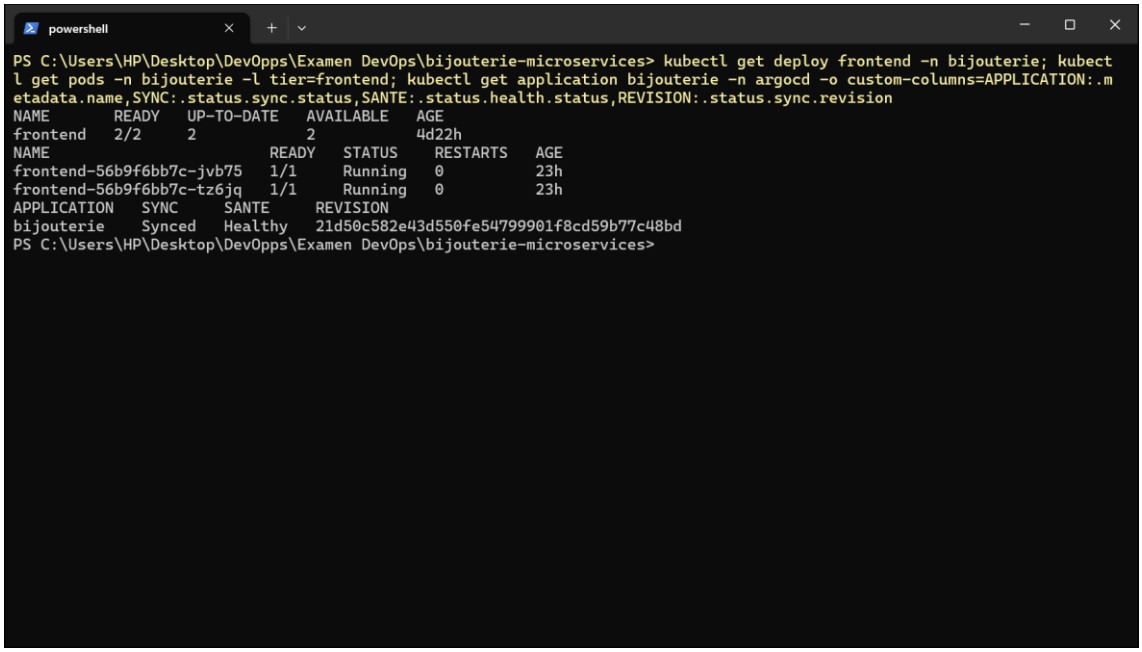
Capture 13a. Application Synced et Healthy.

L'application suit le dépôt, la branche main et le dossier k8s. Elle est synchronisée et en bonne santé.



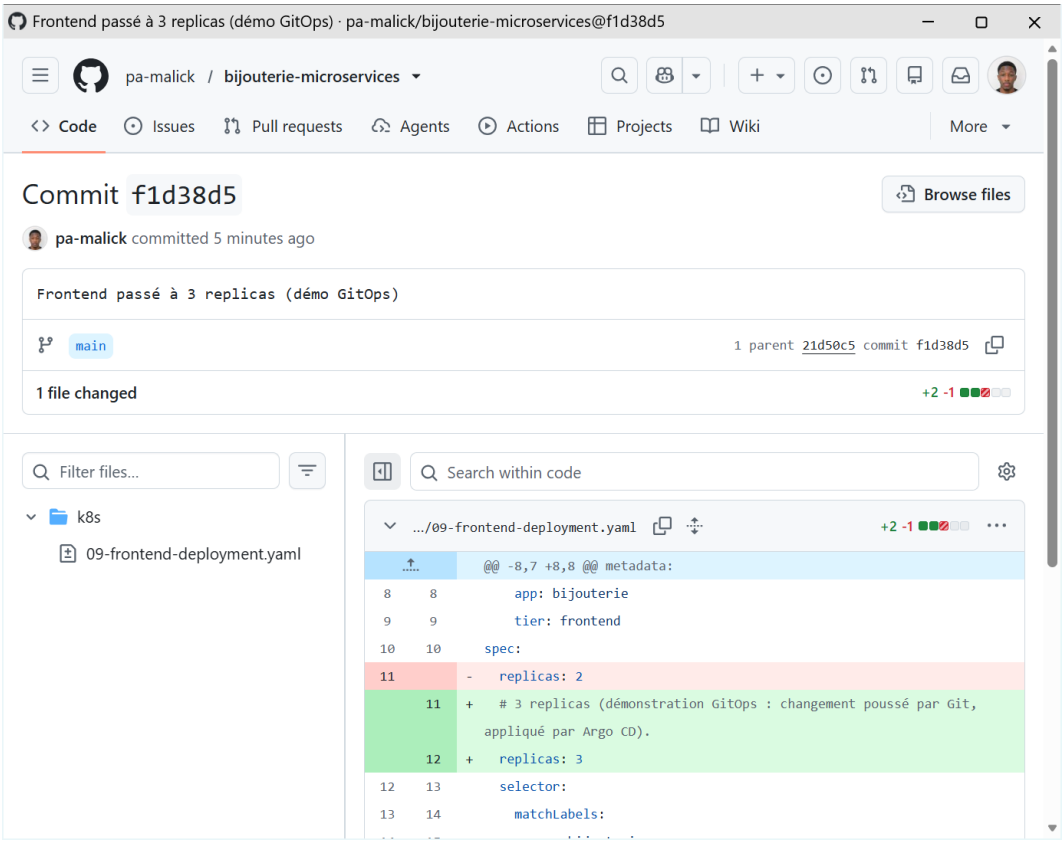
Capture 13b. Arbre des ressources.

Argo CD affiche les ressources déployées : namespace, PVC, Deployments, ReplicaSets et pods.



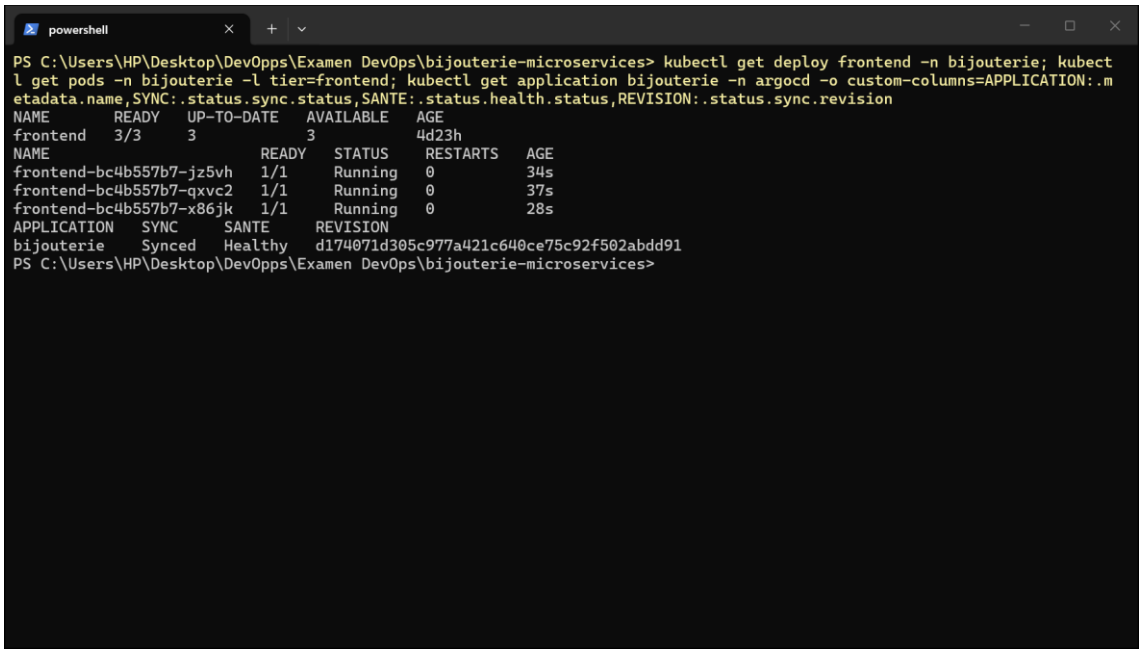
Capture 14a. Avant le commit : frontend à 2 replicas.

Le Deployment frontend a 2 pods prêts. L'application est synchronisée sur la révision 21d50c5.



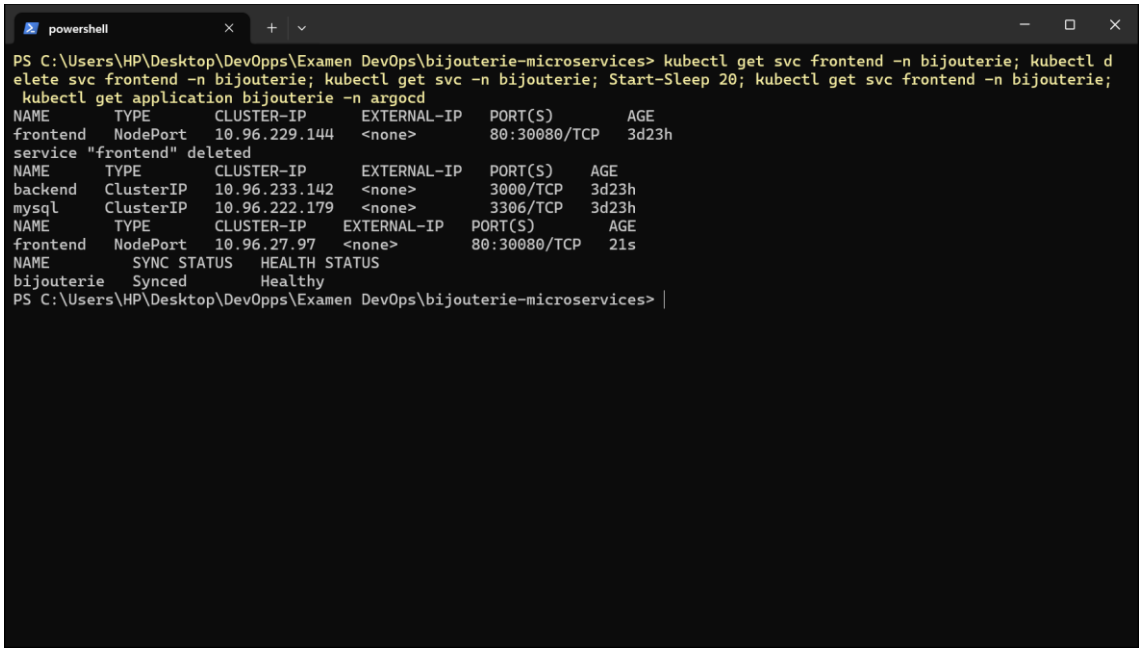
Capture 14b. Le commit qui passe le frontend à 3 replicas.

Une seule ligne change dans k8s/09-frontend-deployment.yaml : replicas passe de 2 à 3. Aucune commande kubectl n'est lancée.



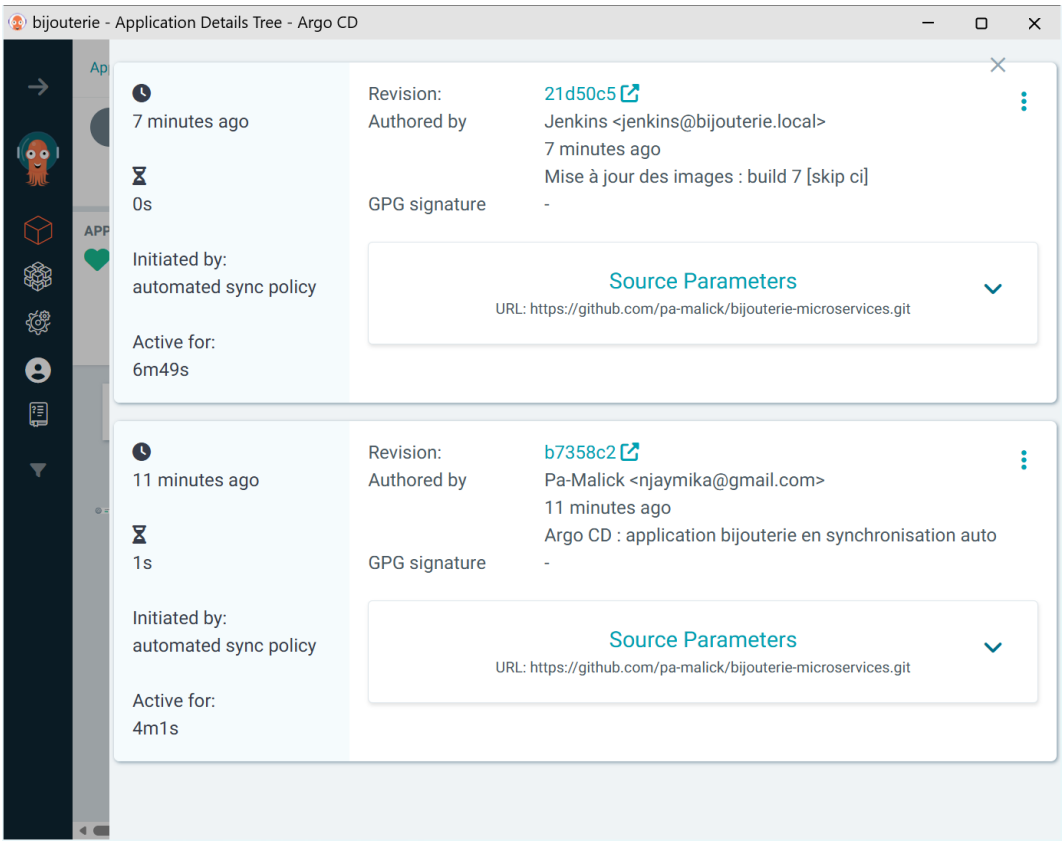
Capture 14c. Après la synchronisation : frontend à 3 replicas.

Le Deployment a 3 pods prêts et l'application est de nouveau Synced et Healthy. Elle prouve qu'Argo CD a appliqué seul le changement poussé dans Git. Entre-temps, Jenkins avait aussi poussé les tags 9 : Argo CD a appliqué les deux commits en une seule synchronisation (révision d174071).



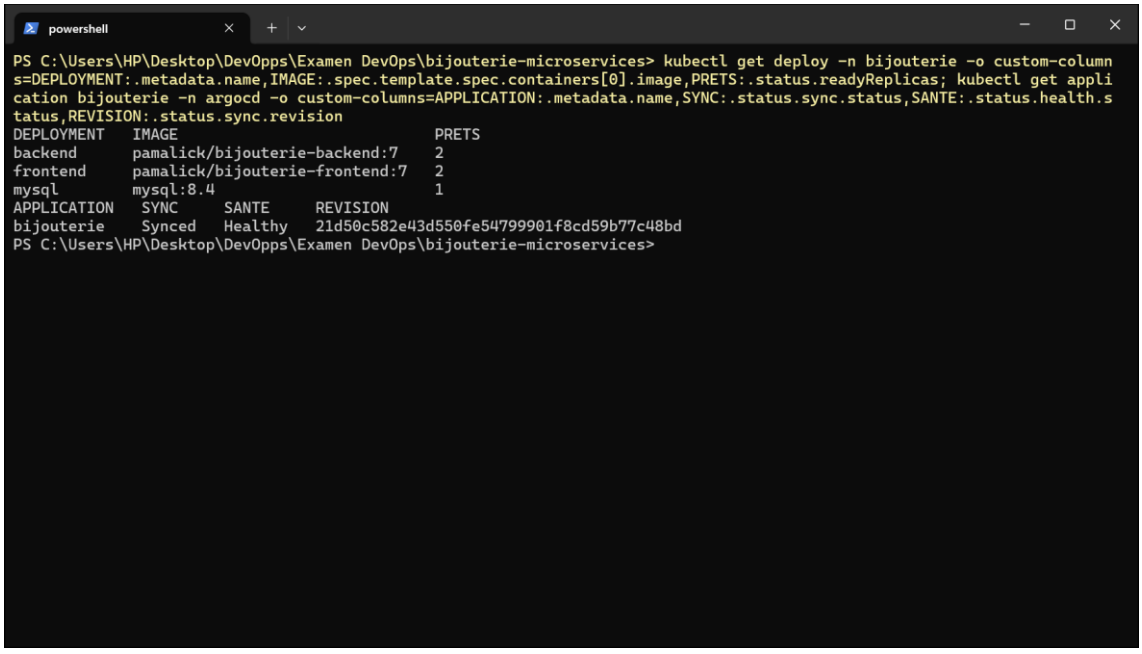
Capture 15. Auto-réparation après suppression du Service frontend.

Le Service est supprimé avec kubectl. Vingt secondes plus tard, il existe de nouveau avec une nouvelle adresse et un âge de 21 s. Elle prouve le selfHeal : Argo CD remet l'état décrit dans Git.



Capture 16a. Historique des synchronisations.

Deux synchronisations automatiques : le commit de l'Application (tags 4), puis le commit du build Jenkins n°7 (tags 7).



Capture 16b. Images du build n°7 déployées.

Les Deployments utilisent le tag 7 sans aucune commande manuelle. Elle prouve la chaîne complète : commit, build, push, mise à jour des manifests, déploiement.

7. Sécurité

Les Secrets de ce TP sont committés dans Git, encodés en base64. Base64 est un encodage, pas un chiffrement : n'importe qui peut lire la valeur avec base64 -d. C'est toléré ici, mais pas en production.

Ce qui est déjà en place :

- Le fichier .env est exclu du dépôt.
- Les tokens Docker Hub et GitHub sont des credentials Jenkins, masqués dans les logs.
- Le token GitHub est limité à un seul dépôt et à la seule permission nécessaire.
- La base et l'API ne sont pas exposées hors du cluster.

En production, il faudrait sortir les valeurs secrètes de Git :

Outil	**Principe**
Sealed Secrets	Le Secret est chiffré avec une clé publique ; seul le contrôleur du cluster peut le déchiffrer. Le fichier chiffré peut aller dans Git.
External Secrets Operator	Le cluster lit les valeurs dans un coffre externe et crée les Secrets.
HashiCorp Vault	Coffre centralisé, avec droits d'accès, audit et rotation des secrets.
SOPS	Chiffre les valeurs dans les fichiers YAML ; Argo CD les déchiffre au déploiement.

Il faudrait aussi activer le chiffrement des Secrets au repos dans etcd et limiter leur lecture par RBAC.

8. Difficultés rencontrées

Problème	**Diagnostic**	**Solution**
git push : erreur 400	Les chevrons de l'exemple étaient restés dans l'URL du remote	git remote set-url origin avec la bonne URL
docker push : denied: requested access to the resource is denied	Aucun identifiant Docker Hub enregistré	docker login avec un Access Token
docker login : net/http: TLS handshake timeout	Accès intermittent du moteur Docker à auth.docker.io	Redémarrage de Docker Desktop et nouvelles tentatives
docker login : error storing credentials	Le coffre d'identifiants de Docker Desktop échouait	Dossier de configuration séparé : docker --config
Nœud kind qui ne démarre pas	Versions récentes incompatibles avec les cgroups v1 du poste	Image kindest/node:v1.32.2 épinglée
Jenkins : permission denied sur /var/run/docker.sock	Le socket appartient au groupe root, l'utilisateur jenkins n'en fait pas partie	Relance du conteneur avec --group-add 0
Build n°3 : Could not find credentials entry with ID github-credentials	Build parti avant la création du credential	Création du credential, puis nouveau build
Build n°5 : remote rejected, cannot lock ref refs/heads/main	Deux builds lancés en même temps sur le même commit	Rien à corriger : le build n°4 avait poussé
Builds en boucle (risque)	Le Poll SCM voit aussi les commits de Jenkins	Message [skip ci] et condition when
Interface Argo CD inaccessible	Le kubectl port-forward s'était arrêté	Port-forward relancé dans une boucle
Accents mal encodés dans un bijou ajouté par curl	curl sous Windows n'envoie pas le texte en UTF-8	Correction par une requête PUT

La démarche a été la même à chaque fois : lire le message d'erreur exact, vérifier l'état réel (kubectl get, logs, console Jenkins), puis corriger une seule chose à la fois.

9. Conclusion

L'application tourne sur Kubernetes avec une configuration externalisée, des Services adaptés à chaque rôle et un volume persistant pour la base. Jenkins construit et publie les images à chaque commit, puis met à jour les manifests. Argo CD déploie ces changements seul et corrige toute modification manuelle.

Le point le plus formateur a été la boucle complète : un simple commit suffit à produire et déployer une nouvelle version. Les prochaines étapes seraient de chiffrer les Secrets et de remplacer le Poll SCM par un webhook GitHub.